

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống thông minh hỗ trợ việc quản lý khách sạn

LƯU VIỆT HOÀN

Hoan.LV215054@sis.hust.edu.vn

Chương trình đào tạo: Việt-Nhật (IT-E6)

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2025

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc tới giáo viên hướng dẫn - TS.Trần Vĩnh Đức. Thầy đã luôn tận tâm chỉ dẫn, gợi ý và định hướng cho em trong quá trình thực hiện đồ án. Nhờ có sự trợ giúp ấy mà em đã có thể thuận lợi phát triển và hoàn thành đồ án.

Em cũng xin gửi lời cảm ơn chân thành tới các thầy cô của Trường Công Nghệ Thông Tin và Truyền Thông. Với sự tận tâm và nhiệt huyết, các thầy cô đã truyền dạy cho em những nền tảng kiến thức vững vàng để thực hiện đồ án.

Và không thể không gửi lời cảm ơn tới gia đình và người thân, những người luôn quan tâm, động viên và yêu thương. Đó là nguồn động lực to lớn, là chỗ dựa tinh thần để em vượt qua những khó khăn trong quá trình thực hiện đồ án.

Lời cuối, em xin gửi lời cảm ơn tới những người bạn đã đi bên em, giúp đỡ, động viên và góp ý trong suốt khoảng thời gian làm đồ án.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh ngành du lịch và khách sạn phát triển mạnh, nhu cầu quản lý hiệu quả hoạt động kinh doanh, đồng thời nâng cao trải nghiệm khách hàng ngày càng được quan tâm. Tuy nhiên, nhiều hệ thống đặt phòng hiện nay mới chỉ dừng lại ở việc quản lý thông tin và đơn đặt phòng đơn thuần, chưa khai thác được tiềm năng của dữ liệu và trí tuệ nhân tạo để hỗ trợ ra quyết định.

Để khắc phục vấn đề trên, em lựa chọn tiếp cận theo hướng xây dựng một hệ thống quản lý và đặt phòng khách sạn tích hợp mô hình học máy để dự đoán lượng khách hàng theo ngày. Hướng tiếp cận này không chỉ nâng cao hiệu quả quản lý, mà còn hỗ trợ khách sạn chủ động trong việc lên kế hoạch về nhân sự, dịch vụ và khuyến mãi.

Hệ thống cho phép quản lý thông tin khách sạn, phòng, nhân viên, dịch vụ, đơn đặt phòng, mã giảm giá. Đồng thời thống kê doanh thu và đưa ra dự đoán lượng khách hàng ngày. Ngoài ra, việc đặt phòng cũng trở nên đơn giản và hiệu quả hơn khi có thể đặt trực tiếp tại lễ tân hoặc đặt online và thanh toán online.

Đóng góp chính của đồ án là một hệ thống hoàn chỉnh, dễ triển khai thực tế, có khả năng mở rộng thêm nhiều tính năng học máy khác trong tương lai. Kết quả đạt được là một ứng dụng hoạt động ổn định, có độ chính xác dự đoán chấp nhận được và hỗ trợ rõ ràng cho người quản lý khách sạn trong việc đưa ra quyết định.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 Khảo sát từ người dùng/khách hàng	5
2.1.2 Khảo sát từ các phần mềm tương tự	5
2.1.3 Các tính năng cần phát triển.....	6
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã Quản lý phòng	9
2.2.3 Biểu đồ use case phân rã Quản dịch vụ	10
2.2.4 Biểu đồ use case phân rã Quản lý mã giảm giá.....	11
2.2.5 Biểu đồ use case phân rã Quản lý đơn đặt của khách hàng	12
2.2.6 Biểu đồ use case phân rã Quản lý nhân viên	13
2.2.7 Biểu đồ use case phân rã Quản lý tài khoản	14
2.2.8 Biểu đồ use case phân rã Quản lý đơn đặt.....	15
2.2.9 Quy trình nghiệp vụ	15
2.3 Đặc tả chức năng	17
2.3.1 Đặc tả use case Đặt phòng	17
2.3.2 Đặc tả use case Xem dự đoán lượng khách	18
2.3.3 Đặc tả use case Thêm đơn đặt phòng.....	18

2.3.4 Đặc tả use case Thêm mã giảm giá	19
2.4 Yêu cầu phi chức năng	19
2.5 Tổng kết.....	20
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	21
3.1 React.js.....	21
3.1.1 Ưu điểm	21
3.1.2 Nhược điểm.....	21
3.1.3 Ứng dụng trong đồ án	21
3.1.4 Lý do lựa chọn	21
3.2 Node.js & Express.js.....	22
3.2.1 Ưu điểm	22
3.2.2 Nhược điểm.....	22
3.2.3 Ứng dụng trong đồ án	22
3.2.4 Lý do lựa chọn	22
3.3 MySQL	23
3.3.1 Ưu điểm	23
3.3.2 Nhược điểm.....	23
3.3.3 Ứng dụng trong đồ án	23
3.3.4 Lý do lựa chọn	23
3.4 Sequelize ORM	23
3.4.1 Ưu điểm	24
3.4.2 Nhược điểm.....	24
3.4.3 Ứng dụng trong đồ án	24
3.4.4 Lý do lựa chọn	24
3.5 Flask (Python).....	24
3.5.1 Ưu điểm	24

3.5.2 Nhược điểm.....	24
3.5.3 Ứng dụng trong đồ án.....	25
3.5.4 Lý do lựa chọn.....	25
3.6 Random Forest.....	25
3.6.1 Ưu điểm.....	25
3.6.2 Nhược điểm.....	25
3.6.3 Ứng dụng trong đồ án.....	25
3.6.4 Lý do lựa chọn.....	25
3.7 XGBoost.....	26
3.7.1 Ưu điểm.....	26
3.7.2 Nhược điểm.....	26
3.7.3 Ứng dụng trong đồ án.....	26
3.7.4 Lý do lựa chọn.....	26
3.8 SARIMA.....	26
3.8.1 Ưu điểm.....	26
3.8.2 Nhược điểm.....	26
3.8.3 Ứng dụng trong đồ án.....	27
3.8.4 Lý do lựa chọn.....	27
3.9 Tổng kết.....	27
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	28
4.1 Thiết kế kiến trúc.....	28
4.1.1 Lựa chọn kiến trúc phần mềm.....	28
4.1.2 Thiết kế tổng quan.....	30
4.1.3 Thiết kế chi tiết gói.....	33
4.2 Thiết kế chi tiết.....	34
4.2.1 Thiết kế giao diện.....	34

4.2.2 Thiết kế lớp	39
4.2.3 Thiết kế cơ sở dữ liệu	43
4.2.4 Thiết kế module học máy	53
4.3 Xây dựng ứng dụng.....	58
4.3.1 Thư viện và công cụ sử dụng	58
4.3.2 Kết quả đạt được	59
4.3.3 Minh họa các chức năng chính	61
4.4 Kiểm thử.....	65
4.5 Triển khai	66
4.5.1 Mô hình triển khai.....	66
4.5.2 Thiết bị triển khai thử nghiệm	67
4.5.3 Kết quả triển khai.....	67
4.6 Tổng kết.....	68
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	69
5.1 Nghiệp vụ cho hệ thống booking và quản lý khách sạn	69
5.1.1 Giới thiệu bài toán.....	69
5.1.2 Giải pháp	69
5.1.3 Kết quả đạt được	70
5.2 Ứng dụng học máy trong dự đoán lượng khách	70
5.2.1 Giới thiệu bài toán.....	70
5.2.2 Giải pháp	71
5.2.3 Kết quả đạt được	72
5.3 Áp dụng các nguyên tắc thiết kế vào giao diện người dùng	72
5.3.1 Giới thiệu vấn đề.....	72
5.3.2 Giải pháp thiết kế giao diện.....	72
5.3.3 Kết quả đạt được	73

5.4 Tổng kết.....	73
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	74
6.1 Kết luận	74
6.1.1 Tự đánh giá kết quả thực hiện đồ án	74
6.1.2 Đóng góp nổi bật	75
6.1.3 Bài học kinh nghiệm rút ra.....	75
6.2 Hướng phát triển.....	75
6.2.1 Hoàn thiện các chức năng hiện tại.....	75
6.2.2 Nâng cấp và mở rộng chức năng	76
6.2.3 Tổng kết	76

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	7
Hình 2.2	Biểu đồ use case phân rã Quản lý phòng	9
Hình 2.3	Biểu đồ use case phân rã Quản dịch vụ	10
Hình 2.4	Biểu đồ use case phân rã Quản lý mã giảm giá	11
Hình 2.5	Biểu đồ use case phân rã Quản lý đơn đặt của khách hàng . . .	12
Hình 2.6	Biểu đồ use case phân rã Quản lý nhân viên	13
Hình 2.7	Biểu đồ use case phân rã Quản lý tài khoản	14
Hình 2.8	Biểu đồ use case phân rã Quản lý đơn đặt	15
Hình 2.9	Biểu đồ hoạn động quy trình nghiệp vụ Đặt phòng	16
Hình 4.1	mô hình MVC	28
Hình 4.2	Biểu đồ gói tổng quan	30
Hình 4.3	Biểu đồ gói chi tiết	33
Hình 4.4	Bố cục màn hình của nhân viên và quản lý	35
Hình 4.5	Bố cục header của màn hình nhân viên và quản lý	36
Hình 4.6	Bố cục sidebar của màn hình của nhân viên và quản lý	36
Hình 4.7	Bố cục màn hình khách hàng	37
Hình 4.8	Bố cục header của màn hình khách hàng	37
Hình 4.9	Màn hình minh họa xem chi tiết đơn đặt của khách hàng . . .	38
Hình 4.10	Màn hình minh họa tìm kiếm khách sạn	38
Hình 4.11	Thiết kế chi tiết lớp BookingController	39
Hình 4.12	Thiết kế chi tiết lớp RoomController	40
Hình 4.13	Thiết kế chi tiết lớp HotelController	41
Hình 4.14	Biểu đồ trình tự use case đặt phòng	42
Hình 4.15	Biểu đồ trình tự use case thêm đơn đặt phòng	43
Hình 4.16	Biểu đồ thực thể liên kết	44
Hình 4.17	Api dự đoán lượng khách	54
Hình 4.18	Api huấn luyện mô hình	54
Hình 4.19	Mẫu dữ liệu sau khi xử lý	55
Hình 4.20	Mô hình Random Forest	56
Hình 4.21	Mô hình XGBoost	56
Hình 4.22	Mô hình Sarima	56
Hình 4.23	Quy trình xử lý dữ liệu trước khi dự đoán	57
Hình 4.24	Giao diện tìm kiếm khách sạn	61
Hình 4.25	Giao diện chi tiết khách sạn	62

Hình 4.26	Giao diện xác nhận đặt phòng	63
Hình 4.27	Giao diện dự đoán lượng khách hàng ngày	64
Hình 4.28	Giao diện duyệt đơn đặt phòng	64
Hình 4.29	Giao diện xem chi tiết đơn đặt phòng của khách hàng	65

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các hệ thống đặt và quản lý khách sạn theo tiêu chí . . .	6
Bảng 2.2	Đặc tả use case Đặt phòng	17
Bảng 2.3	Đặc tả use case Xem dự đoán lượng khách	18
Bảng 2.4	Đặc tả use case Thêm đơn đặt phòng	18
Bảng 2.5	Đặc tả use case Thêm mã giảm giá	19
Bảng 4.1	Thông số kỹ thuật màn hình	35
Bảng 4.2	Mô tả bảng users.	46
Bảng 4.3	Mô tả bảng staffs.	46
Bảng 4.4	Mô tả bảng hotels.	47
Bảng 4.5	Mô tả bảng reviews.	47
Bảng 4.6	Mô tả bảng bookings.	48
Bảng 4.7	Mô tả bảng vouchers.	49
Bảng 4.8	Mô tả bảng rooms.	49
Bảng 4.9	Mô tả bảng room_types.	50
Bảng 4.10	Mô tả bảng amenities.	50
Bảng 4.11	Mô tả bảng hotel_amenities.	50
Bảng 4.12	Mô tả bảng hotel_amenities.	50
Bảng 4.13	Mô tả bảng room_type_amenities.	51
Bảng 4.14	Mô tả bảng services.	51
Bảng 4.15	Mô tả bảng service_bookings.	52
Bảng 4.16	Mô tả bảng revenues.	52
Bảng 4.17	Mô tả bảng predicts.	53
Bảng 4.18	Mô tả bảng user_vouchers.	53
Bảng 4.19	Thư viện và công cụ sử dụng	59
Bảng 4.20	Thống kê thông tin ứng dụng	60
Bảng 4.21	Bảng kiểm thử chức năng thêm đơn đặt phòng	66
Bảng 4.22	Bảng kiểm thử chức năng thêm loại phòng	66
Bảng 4.23	Cấu hình thiết bị triển khai thử nghiệm	67

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, ngành du lịch và lưu trú tại Việt Nam không ngừng tăng trưởng, kéo theo nhu cầu quản lý khách sạn một cách hiệu quả và chuyên nghiệp. cạnh tranh ngày càng cao trong lĩnh vực này khiến các doanh nghiệp phải liên tục nâng cao chất lượng dịch vụ, tối ưu quy trình vận hành và chủ động trong việc tiếp cận khách hàng. Tuy nhiên, nhiều khách sạn vừa và nhỏ hiện vẫn đang quản lý đặt phòng một cách thủ công hoặc sử dụng các phần mềm rời rạc, thiếu tính liên kết và khả năng phân tích dữ liệu.

Bên cạnh đó, một trong những thách thức lớn mà các khách sạn thường gặp phải là việc không thể dự báo trước được lượng khách theo ngày để chủ động trong điều phối nhân sự, chuẩn bị dịch vụ hay đưa ra các chương trình khuyến mãi phù hợp. Việc thiếu công cụ hỗ trợ ra quyết định dựa trên dữ liệu không chỉ gây lãng phí nguồn lực mà còn ảnh hưởng đến doanh thu và trải nghiệm của khách hàng.

Ngoài ra, xu hướng của khách hàng hiện nay là ưu tiên các dịch vụ nhanh chóng, tiện lợi, đặc biệt là khả năng đặt phòng trực tuyến một cách dễ dàng, không mất thời gian chờ đợi hay trải qua các quy trình rườm rà. Trong bối cảnh cạnh tranh gay gắt, nếu các khách sạn không bắt kịp xu hướng này, họ sẽ khó giữ chân khách hàng và có nguy cơ bị tụt lại phía sau trên thị trường.

Nếu vấn đề này được giải quyết tốt, nó không chỉ giúp các khách sạn quản lý hiệu quả hơn và đưa ra quyết định chính xác, mà còn mang lại trải nghiệm thuận tiện cho khách hàng trong việc đặt phòng – từ đó góp phần gia tăng số lượng đặt phòng và cải thiện doanh thu.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, trên thị trường đã xuất hiện nhiều hệ thống hỗ trợ quản lý và đặt phòng khách sạn trực tuyến, từ các nền tảng lớn như Booking.com, Agoda cho đến các phần mềm quản lý nội bộ được phát triển riêng cho từng khách sạn hoặc chuỗi khách sạn. Các hệ thống này thường cung cấp những chức năng cơ bản như quản lý thông tin phòng, theo dõi tình trạng đặt phòng, xử lý thanh toán và lưu trữ dữ liệu khách hàng. Nhờ đó, các khách sạn có thể giảm thiểu thao tác thủ công, nâng cao hiệu quả vận hành và cải thiện chất lượng dịch vụ.

Tuy nhiên, phần lớn các nền tảng hiện có vẫn tập trung chủ yếu vào các chức năng truyền thống, chưa tận dụng tốt tiềm năng phân tích dữ liệu lịch sử để mang lại giá trị cao hơn trong hoạt động quản lý. Một số nền tảng cao cấp có tích hợp

công cụ phân tích và hỗ trợ ra quyết định, song mức chi phí triển khai và duy trì khá cao, cùng với khả năng tùy biến thấp khiến các khách sạn vừa và nhỏ gặp nhiều khó khăn trong việc tiếp cận và khai thác hiệu quả. Điều này tạo ra một khoảng trống rõ rệt giữa nhu cầu thực tế và các giải pháp hiện hành.

Về phía người dùng, xu hướng chung là ưu tiên sự nhanh chóng, tiện lợi và khả năng tiếp cận dịch vụ mọi lúc, mọi nơi. Do đó, một hệ thống đặt phòng hiện đại cần phải đáp ứng tốt các tiêu chí như giao diện trực quan, dễ sử dụng, tốc độ phản hồi nhanh và khả năng hoạt động ổn định trên nhiều nền tảng khác nhau như trình duyệt web, ứng dụng điện thoại hoặc máy tính bảng. Trải nghiệm người dùng (UX) ngày càng trở thành yếu tố cạnh tranh quan trọng bên cạnh chất lượng dịch vụ.

Mặc dù công nghệ đã có nhiều bước tiến đáng kể, nhưng một số hạn chế vẫn còn tồn tại. Đặc biệt, khả năng dự báo nhu cầu lưu trú – vốn là một trong những yếu tố then chốt giúp khách sạn chủ động trong việc bố trí nhân sự, điều chỉnh giá phòng và lập kế hoạch marketing – vẫn chưa được quan tâm đúng mức. Phần lớn các hệ thống chưa tích hợp mô hình học máy hay trí tuệ nhân tạo để tận dụng dữ liệu lịch sử cho việc dự đoán xu hướng khách hàng. Đồng thời, một số sản phẩm vẫn chưa đầu tư bài bản vào thiết kế giao diện và nâng cao trải nghiệm người dùng, khiến khách hàng dễ bị mất thiện cảm hoặc gặp khó khăn trong quá trình thao tác.

Từ những phân tích trên, em nhận thấy nhu cầu cấp thiết trong việc xây dựng một hệ thống quản lý và đặt phòng khách sạn hiện đại, không chỉ đáp ứng các chức năng quản lý cơ bản mà còn có khả năng dự báo lượng khách hàng theo từng ngày dựa trên dữ liệu lịch sử. Hệ thống này sẽ giúp chủ khách sạn lập kế hoạch vận hành hiệu quả hơn, tối ưu hóa nguồn lực và gia tăng doanh thu trong từng thời điểm. Đồng thời, đối với khách hàng, hệ thống cũng sẽ mang đến trải nghiệm đặt phòng thuận tiện, nhanh chóng và đáng tin cậy. Các chức năng chính của hệ thống bao gồm: quản lý phòng, đặt phòng, quản lý nhân viên, thanh toán, và tích hợp mô hình học máy để dự báo lượng khách.

1.3 Định hướng giải pháp

Để giải quyết bài toán đã đặt ra, em lựa chọn định hướng phát triển một hệ thống web ứng dụng, sử dụng kiến trúc Client-Server. Công nghệ được sử dụng gồm: React cho phần giao diện người dùng nhằm đảm bảo tính tương tác và phản hồi nhanh; Node.js và Express cho backend giúp xử lý nghiệp vụ và kết nối cơ sở dữ liệu MySQL. Đặc biệt, em tích hợp một mô hình học máy sử dụng thuật toán Random Forest, XGBoost và SARIMA, triển khai độc lập bằng Flask và giao tiếp qua API, để dự đoán lượng khách hàng theo ngày dựa trên dữ liệu lịch sử.

Hệ thống giúp khách hàng đặt phòng một cách thuận tiện và dễ dàng, có thể

theo dõi đơn đặt phòng và đặt thêm các dịch vụ của khách sạn. Khách hàng cũng có thể thanh toán online. Bên cạnh đấy, hệ thống giúp nhân viên khách sạn dễ dàng quản lý các đơn đặt phòng, xác nhận đơn, check-in, check-out. Ngoài ra, hệ thống còn cung cấp công cụ dự báo lượng khách, giúp chủ khách sạn có thêm cơ sở ra quyết định vận hành hiệu quả hơn.

Đóng góp chính của đề án là xây dựng được một hệ thống có khả năng vận hành thực tế, tích hợp học máy một cách linh hoạt, dễ mở rộng. Kết quả đạt được là phần mềm hoàn chỉnh với giao diện thân thiện, hiệu suất ổn định và mô hình dự báo có độ chính xác chấp nhận được trong bối cảnh ứng dụng thực tiễn.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức như sau.

Chương 2 tập trung khảo sát thực tế nhu cầu đặt phòng và quản lý khách sạn, phân tích một số hệ thống hiện có để rút ra điểm mạnh – yếu, đồng thời xác định các yêu cầu cần thiết cho hệ thống cần xây dựng. Qua việc thu thập ý kiến từ người dùng và tham khảo các giải pháp công nghệ hiện đại, chương sẽ xác định rõ các chức năng cần thiết. Ngoài ra, chương cũng đưa ra đặc tả yêu cầu chức năng và phi chức năng chi tiết, làm cơ sở cho các bước thiết kế và triển khai hệ thống trong các chương tiếp theo.

Chương 3 trình bày tổng quan về các công nghệ, nền tảng và công cụ được lựa chọn để xây dựng hệ thống, bao gồm cả công nghệ phần mềm và mô hình học máy phục vụ cho việc dự đoán lượng khách. Mỗi công nghệ đều được phân tích về ưu điểm, nhược điểm và khả năng ứng dụng trong bối cảnh bài toán. Việc lựa chọn được đưa ra dựa trên các yêu cầu đã xác định ở Chương 2, đảm bảo tính phù hợp, khả thi và hiệu quả khi triển khai vào hệ thống thực tế.

Chương 4 tập trung trình bày quá trình thiết kế và xây dựng hệ thống quản lý và đặt phòng khách sạn tích hợp học máy. Nội dung bao gồm lựa chọn kiến trúc phần mềm phù hợp, thiết kế các biểu đồ UML để thể hiện cấu trúc hệ thống và mối quan hệ giữa các thành phần. Chương cũng mô tả chi tiết quá trình xây dựng giao diện người dùng, thiết kế cơ sở dữ liệu, tích hợp các mô hình học máy, phát triển các chức năng chính và thực hiện kiểm thử, triển khai hệ thống để đánh giá mức độ đáp ứng yêu cầu thực tế.

Chương 5 trình bày những đóng góp quan trọng trong quá trình thực hiện đề án, bao gồm các giải pháp xử lý bài toán nghiệp vụ đặc thù trong quản lý và đặt phòng khách sạn, ứng dụng các mô hình học máy như Random Forest, XGBoost và SARIMA để dự đoán lượng khách, cũng như thiết kế giao diện người dùng theo

các nguyên tắc chuẩn nhằm nâng cao trải nghiệm. Mỗi nội dung đều được trình bày có hệ thống với phần nêu vấn đề, giải pháp kỹ thuật được đề xuất và kết quả đạt được sau triển khai.

Chương 6 tổng kết quá trình thực hiện đồ án tốt nghiệp, đánh giá những kết quả đã đạt được, các điểm còn hạn chế, cùng những bài học kinh nghiệm rút ra trong suốt quá trình triển khai. Bên cạnh đó, chương cũng đề xuất các hướng phát triển trong tương lai nhằm hoàn thiện hệ thống, bao gồm việc nâng cấp các tính năng hiện có, cải thiện độ chính xác của mô hình học máy, tối ưu hiệu năng hệ thống và mở rộng quy mô phục vụ cho nhiều khách sạn hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này tập trung vào việc khảo sát hiện trạng thực tế liên quan đến bài toán quản lý và đặt phòng khách sạn, bao gồm việc phân tích các hệ thống hiện có, thu thập nhu cầu từ người dùng và đánh giá các xu hướng công nghệ liên quan. Trên cơ sở đó, chương sẽ đưa ra tổng quan về các chức năng chính cần phát triển trong đồ án. Đồng thời, chương cũng tiến hành đặc tả chi tiết các yêu cầu chức năng và phi chức năng, làm cơ sở quan trọng cho việc thiết kế, xây dựng và đánh giá hệ thống phần mềm ở các chương tiếp theo.

2.1 Khảo sát hiện trạng

Để xây dựng một hệ thống quản lý và đặt phòng khách sạn hiệu quả, em tiến hành khảo sát từ ba nguồn chính. Đầu tiên là phản hồi từ người dùng tiềm năng bao gồm chủ khách sạn, nhân viên khách sạn và khách đặt phòng. Tiếp theo là các phần mềm hiện có trên thị trường. Cuối cùng là các hệ thống tương tự.

2.1.1 Khảo sát từ người dùng/khách hàng

Nhóm đối tượng được khảo sát gồm 2 nhóm: khách hàng đặt phòng khách sạn và chủ/nhân viên khách sạn.

a, Khảo sát từ khách hàng

Khách hàng thường là những người có thói quen đặt phòng trực tuyến thông qua các nền tảng như Booking.com, Agoda, hoặc qua website của khách sạn. Nhóm này yêu cầu hệ thống tiện lợi, sử dụng dễ dàng và cung cấp thông tin đầy đủ về các khách sạn và phòng khách sạn. Đồng thời họ mong muốn có thể dễ dàng tìm kiếm khách sạn và có thể thanh toán online.

b, Khảo sát từ chủ/nhân viên khách sạn

Nhóm này có các yêu cầu tập trung vào tính hiệu quả trong quản lý vận hành. Họ cần một hệ thống có khả năng quản lý đầy đủ các thông tin về phòng, loại phòng, nhân viên, và trạng thái phòng theo thời gian thực. Việc phân quyền người dùng theo vai trò cũng rất quan trọng để đảm bảo tính bảo mật và phù hợp với quy trình làm việc. Ngoài ra, hệ thống cần cung cấp các chức năng thống kê, báo cáo doanh thu Đặc biệt, nhóm này thể hiện sự quan tâm đến việc tích hợp mô hình dự báo lượng khách theo ngày nhằm chủ động trong việc bố trí nhân lực, điều chỉnh giá và tối ưu hóa doanh thu.

2.1.2 Khảo sát từ các phần mềm tương tự

Để có cái nhìn tổng quan về thị trường cũng như học hỏi từ các mô hình đã có, em đã tiến hành khảo sát một số hệ thống quản lý và đặt phòng khách sạn phổ biến

hiện nay, bao gồm các nền tảng đặt phòng trực tuyến như Booking.com, Agoda, Traveloka và phần mềm quản lý nội bộ như OHOTEL.

Tiêu chí	Booking.com	Agoda	Traveloka	OHOTEL
Đối tượng chính	Người đặt phòng	Người đặt phòng	Người đặt phòng	Chủ và nhân viên khách sạn
Đặt phòng trực tuyến	Rất tốt	Rất tốt	Rất tốt	Không hỗ trợ
Quản lý nhân viên	Không hỗ trợ	Không hỗ trợ	Không hỗ trợ	Có
Thanh toán trực tuyến	Hỗ trợ nhiều hình thức	Hỗ trợ nhiều hình thức	Hỗ trợ nhiều hình thức	Có tích hợp với các cổng thanh toán
Dự báo lượng khách hàng	Không hỗ trợ	Không hỗ trợ	Không hỗ trợ	Chưa tích hợp
Tích hợp hệ thống khác (CRM, POS)	Không	Không	Không	Có (tùy gói triển khai)
Báo cáo, thống kê nội bộ	Không hỗ trợ	Không hỗ trợ	Không hỗ trợ	Có
Chi phí sử dụng với khách sạn	Miễn phí tham gia – trừ hoa hồng cao	Miễn phí tham gia – trừ hoa hồng cao	Miễn phí tham gia – trừ hoa hồng cao	Gói phí theo tháng/năm
Khả năng tùy biến	Không	Không	Không	Giới hạn theo cấu hình phần mềm
Ưu điểm nổi bật	- Phổ biến toàn cầu - Giao diện đẹp	- Có nhiều chương trình khuyến mãi	- Hỗ trợ dịch vụ đa lĩnh vực - UX tốt	- Quản lý nội bộ toàn diện
Hạn chế chính	- Không hỗ trợ quản lý nội bộ	- Hoa hồng cao cho khách sạn	- Chủ yếu phục vụ người dùng cuối	- Giao diện kỹ thuật, khó tùy biến sâu

Bảng 2.1: So sánh các hệ thống đặt và quản lý khách sạn theo tiêu chí

2.1.3 Các tính năng cần phát triển

Dựa trên quá trình khảo sát và bảng so sánh trên (Bảng 2.1), có thể thấy mỗi hệ thống đều có điểm mạnh riêng nhưng cũng tồn tại những hạn chế nhất định, đặc biệt là sự thiếu kết nối giữa trải nghiệm người dùng và quản lý nội bộ khách sạn. Từ đó, hệ thống đề xuất trong đồ án cần tích hợp cả hai khía cạnh này, đồng thời bổ sung khả năng dự báo nhu cầu lưu trú để hỗ trợ quản lý hiệu quả hơn. Các tính năng cần phát triển như sau:

- Quản lý phòng/loại phòng:** Cập nhật trạng thái phòng theo thời gian thực, thay đổi thông tin phòng nếu cần.
- Quản lý dịch vụ:** Thêm, cập nhật các dịch vụ, tiện ích mà khách sạn hỗ trợ.
- Đặt phòng:** Tìm kiếm khách sạn, phòng phù hợp và đặt phòng online. Đồng

thời hỗ trợ thanh toán trực tuyến.

4. **Quản lý đơn đặt phòng:** Quản lý các đơn đặt phòng, xác nhận đơn, check-in, check-out cho khách.
5. **Dự đoán lượng khách hàng ngày:** Đưa ra dự đoán về lượng khách hàng ngày để hỗ trợ chủ khách sạn chủ động trong việc phân bổ nhân sự, sắp xếp phòng ở hợp lý và chuẩn bị các nguồn lực cần thiết, từ đó nâng cao hiệu quả vận hành và giảm thiểu tình trạng quá tải hoặc lãng phí.
6. **Thống kê doanh thu:** Doanh thu sẽ được hệ thống thống kê theo từng ngày, tháng hoặc năm, giúp chủ khách sạn có cái nhìn tổng quan về tình hình kinh doanh trong các giai đoạn khác nhau. Nhờ đó, họ có thể đánh giá hiệu quả hoạt động, phát hiện xu hướng tăng giảm theo mùa và đưa ra các quyết định điều chỉnh chiến lược kịp thời để tối ưu hóa lợi nhuận.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát

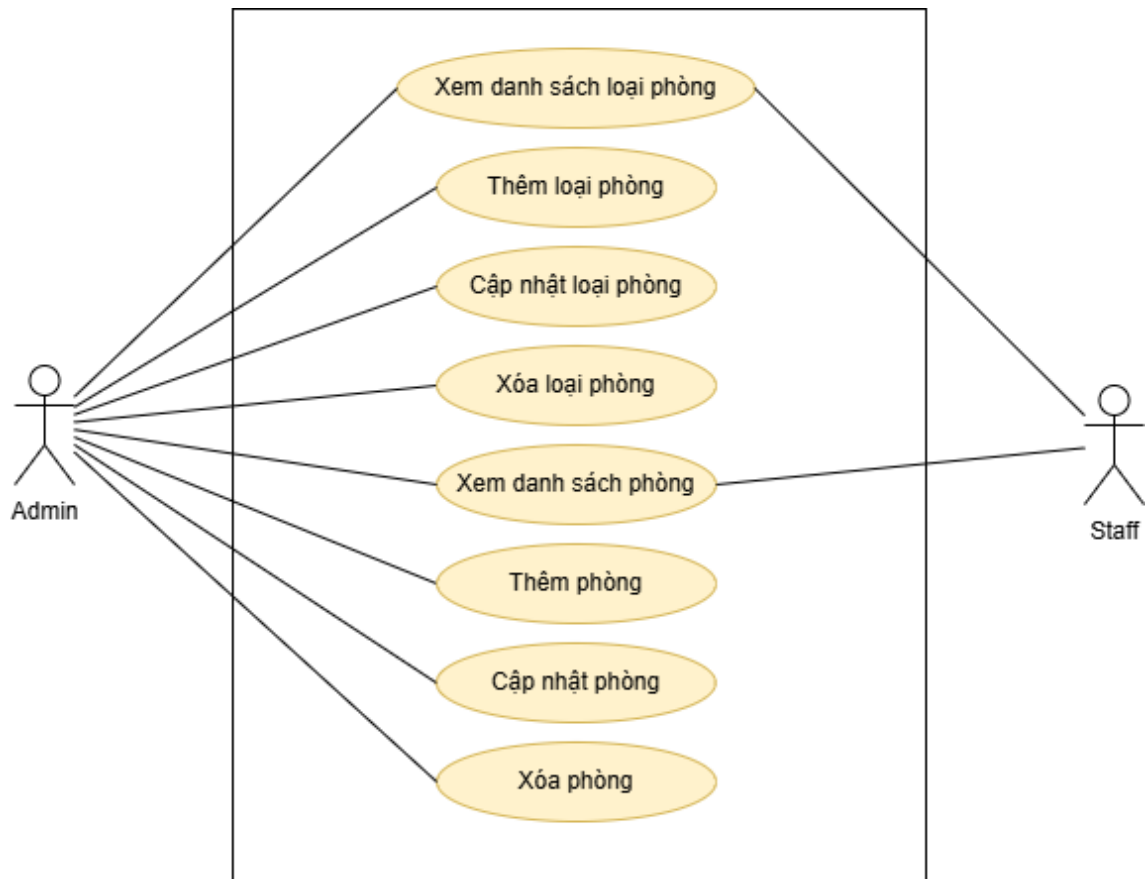
Biểu đồ use case tổng quát gồm có 3 tác nhân chính là **Staff**, **Admin** và **Customer**. **Staff (Nhân viên)** là những người làm việc tại khách sạn (thường là lễ tân),

chịu trách nhiệm vận hành và quản lý các hoạt động hàng ngày liên quan đến khách sạn và khách hàng. **Admin (Quản trị viên)** là người có quyền cao nhất trong hệ thống, thường là quản lý hoặc chủ khách sạn. Họ có quyền truy cập và quản lý gần như tất cả các chức năng, bao gồm cả việc quản lý nhân viên và thông tin chung của khách sạn. **Customer(Khách hàng)** là những người sử dụng dịch vụ của khách sạn, tức là những người có nhu cầu đặt phòng và các dịch vụ đi kèm.

Biểu đồ này gồm 12 use case chính như sau:

1. **Quản lý phòng:** Bao gồm các thao tác như thêm phòng mới, chỉnh sửa thông tin phòng và cập nhật trạng thái phòng theo thời gian thực.
2. **Quản lý dịch vụ:** Thêm, xóa, cập nhật các dịch vụ, tiện ích mà khách sạn hỗ trợ.
3. **Quản lý mã giảm giá:** Thêm, chỉnh sửa các mã giảm giá, chương trình khuyến mãi để áp dụng cho các đơn đặt phòng hoặc dịch vụ.
4. **Quản lý đơn đặt của khách hàng:** Quản lý các đơn đặt phòng, xác nhận đơn, hủy đơn, check-in, check-out cho khách.
5. **Quản lý nhân viên:** Dành cho Admin, cho phép thêm mới, xem và xóa các tài khoản nhân viên.
6. **Chỉnh sửa thông tin khách sạn:** Cho phép Admin cập nhật các thông tin chung về khách sạn như địa chỉ, số điện thoại, mô tả, hình ảnh...
7. **Xem thống kê doanh thu:** Cho phép Admin xem thống kê doanh thu dưới dạng biểu đồ theo từng ngày, tháng hoặc năm, giúp Admin có cái nhìn tổng quan về tình hình kinh doanh trong các giai đoạn khác nhau.
8. **Xem dự đoán lượng khách:** Đưa ra dự đoán về lượng khách hàng ngày giúp khách sạn lập kế hoạch về nhân sự và nguồn lực.
9. **Quản lý tài khoản:** Cho phép đăng nhập vào hệ thống và cập nhật thông tin cá nhân.
10. **Đặt phòng:** Chức năng cốt lõi cho phép khách hàng tìm kiếm khách sạn, các phòng trống, chọn loại phòng, thời gian lưu trú và hoàn tất quá trình đặt phòng.
11. **Quản lý đơn đặt:** Cho phép khách hàng xem lại danh sách các đơn đặt phòng của mình, kiểm tra trạng thái và thực hiện các thao tác như đặt thêm dịch vụ, thanh toán.
12. **Đánh giá khách sạn:** Cho phép người dùng nhận xét và đánh giá về khách sạn sau khi hoàn thành dịch vụ.

2.2.2 Biểu đồ use case phân rã Quản lý phòng



Hình 2.2: Biểu đồ use case phân rã Quản lý phòng

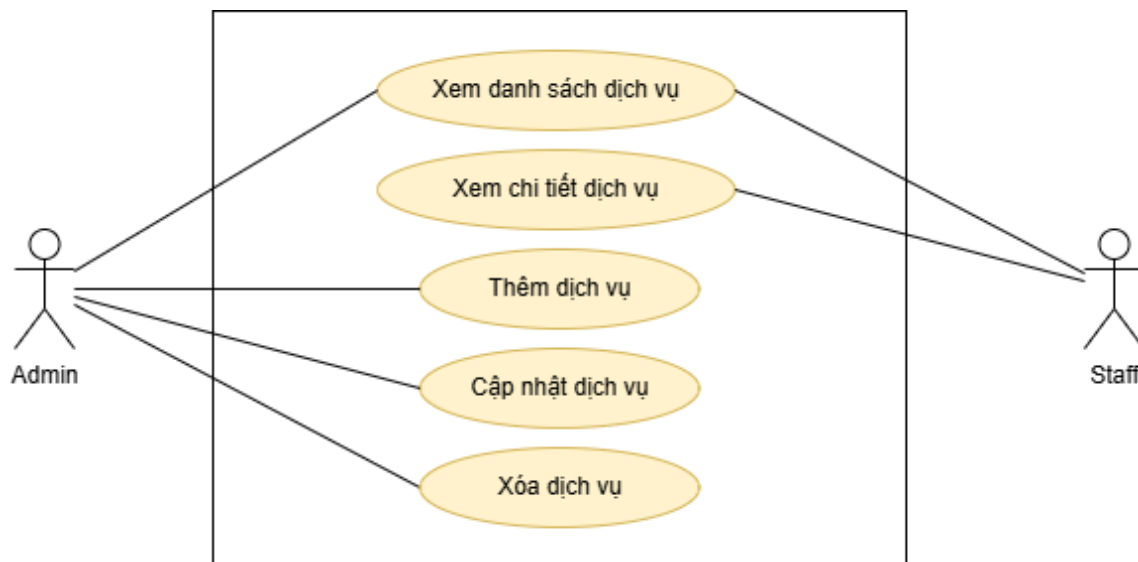
Biểu đồ use case phân rã Quản lý được phân rã thành các use case với hai tác nhân là Admin và Staff như sau:

- **Xem danh sách loại phòng:** Cho phép Admin và Staff xem toàn bộ danh sách các loại phòng hiện có trong khách sạn.
- **Thêm loại phòng:** Cho phép Admin tạo mới một loại phòng với các thông tin chi tiết.
- **Cập nhật loại phòng:** Cho phép Admin chỉnh sửa thông tin của một loại phòng đã tồn tại.
- **Xóa loại phòng:** Cho phép Admin loại bỏ một loại phòng khỏi hệ thống nếu loại phòng đó không còn đơn đặt.
- **Xem danh sách phòng:** Cho phép Admin và Staff xem toàn bộ danh sách các phòng cụ thể trong khách sạn cùng với trạng thái của chúng.
- **Thêm phòng:** Cho phép Admin thêm một phòng vật lý mới vào hệ thống.
- **Cập nhật phòng:** Cho phép Admin chỉnh sửa thông tin hoặc trạng thái của

một phòng cụ thể.

- **Xóa phòng:** Cho phép Admin loại bỏ một phòng cụ thể khỏi hệ thống nếu phòng đó không có đơn đặt.

2.2.3 Biểu đồ use case phân rã Quản dịch vụ

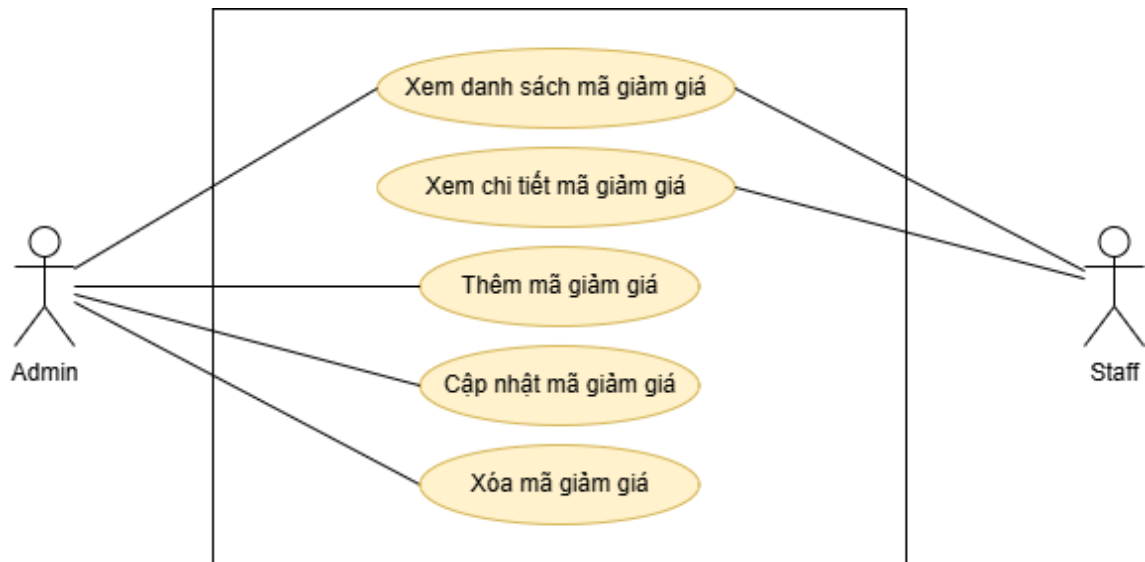


Hình 2.3: Biểu đồ use case phân rã Quản dịch vụ

Biểu đồ use case phân rã Quản lý dịch vụ được phân rã thành các use case với hai tác nhân là Admin và Staff như sau:

- **Xem danh sách dịch vụ:** Cho phép Admin và Staff xem toàn bộ danh sách các dịch vụ hiện có trong khách sạn.
- **Xem chi tiết dịch vụ:** Cho phép Staff xem thông tin chi tiết của một dịch vụ cụ thể.
- **Thêm dịch vụ:** Cho phép Admin tạo mới một dịch vụ với các thông tin chi tiết.
- **Cập nhật dịch vụ:** Cho phép Admin chỉnh sửa thông tin của một dịch vụ đã tồn tại.
- **Xóa dịch vụ:** Cho phép Admin loại bỏ một dịch vụ khỏi hệ thống nếu dịch vụ đó không còn được cung cấp hoặc không có đơn đặt liên quan.

2.2.4 Biểu đồ use case phân rã Quản lý mã giảm giá

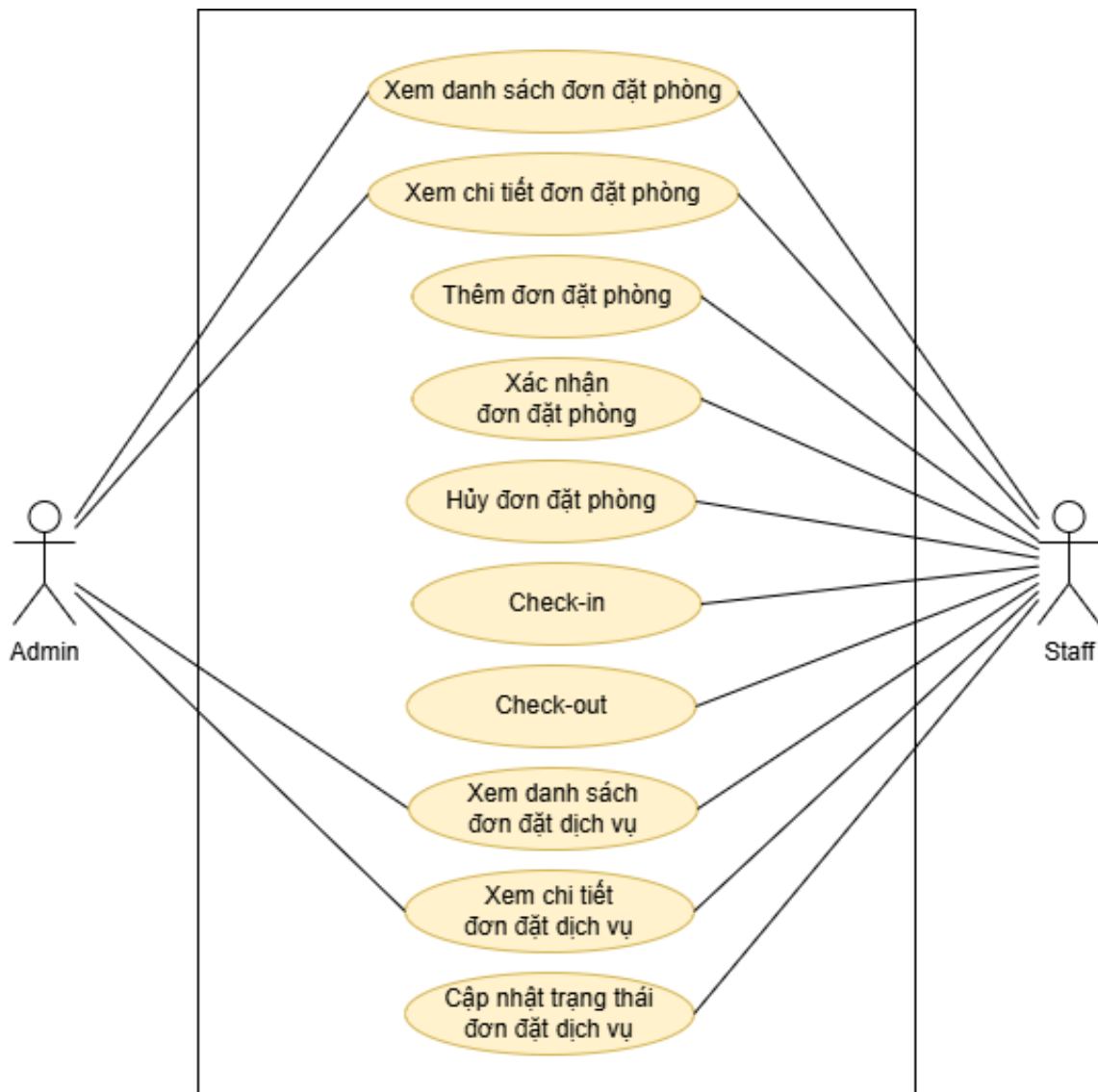


Hình 2.4: Biểu đồ use case phân rã Quản lý mã giảm giá

Biểu đồ use case phân rã Quản lý mã giảm giá được phân rã thành các use case với hai tác nhân là Admin và Staff như sau:

- **Xem danh sách mã giảm giá:** Cho phép Admin và Staff xem toàn bộ danh sách các mã giảm giá hiện có của khách sạn.
- **Xem chi tiết mã giảm giá:** Cho phép Staff xem thông tin chi tiết của một mã giảm giá cụ thể.
- **Thêm mã giảm giá:** Cho phép Admin tạo mới một mã giảm giá với các thông tin chi tiết.
- **Cập nhật mã giảm giá:** Cho phép Admin chỉnh sửa thông tin của một mã giảm giá.
- **Xóa mã giảm giá:** Cho phép Admin loại bỏ một mã giảm giá khỏi hệ thống nếu mã đó không còn hiệu lực hoặc không còn được sử dụng.

2.2.5 Biểu đồ use case phân rã Quản lý đơn đặt của khách hàng



Hình 2.5: Biểu đồ use case phân rã Quản lý đơn đặt của khách hàng

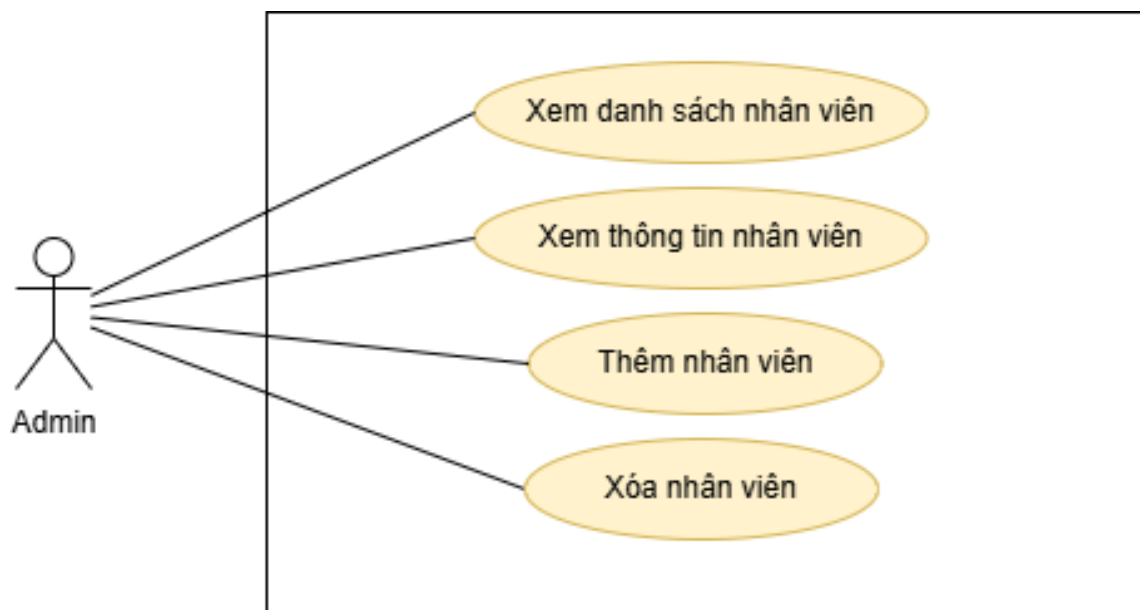
Biểu đồ use case phân rã Quản lý đơn đặt của khách hàng được phân rã thành các use case với hai tác nhân là Admin và Staff như sau:

- **Xem danh sách đơn đặt phòng:** Cho phép Admin và Staff xem toàn bộ danh sách các đơn đặt phòng của khách sạn.
- **Xem chi tiết đơn đặt phòng:** Cho phép Admin và Staff xem thông tin chi tiết của một đơn đặt phòng cụ thể bao gồm thông tin khách hàng, thông tin đặt phòng và các dịch vụ đi kèm.
- **Thêm đơn đặt phòng:** Cho phép Staff tạo mới một đơn đặt phòng thủ công khi khách hàng đặt phòng trực tiếp.
- **Xác nhận đơn đặt phòng:** Cho phép Staff xác nhận một đơn đặt phòng,

chuyển trạng thái từ chờ xử lý sang đã xác nhận.

- **Hủy đơn đặt phòng:** Cho phép Staff hủy một đơn đặt phòng đã tồn tại.
- **Check-in:** Cho phép Staff thực hiện quy trình nhận phòng cho khách hàng, cập nhật trạng thái phòng và đơn đặt.
- **Check-out:** Cho phép Staff thực hiện quy trình trả phòng cho khách hàng, bao gồm việc thanh toán và cập nhật trạng thái phòng.
- **Xem danh sách đơn đặt dịch vụ:** Cho phép Admin và Staff xem toàn bộ danh sách các đơn đặt dịch vụ.
- **Xem chi tiết đơn đặt dịch vụ:** Cho phép Admin và Staff xem thông tin chi tiết của một đơn đặt dịch vụ cụ thể.
- **Cập nhật trạng thái đơn đặt dịch vụ:** Cho phép Staff cập nhật trạng thái của một đơn đặt dịch vụ.

2.2.6 Biểu đồ use case phân rã Quản lý nhân viên



Hình 2.6: Biểu đồ use case phân rã Quản lý nhân viên

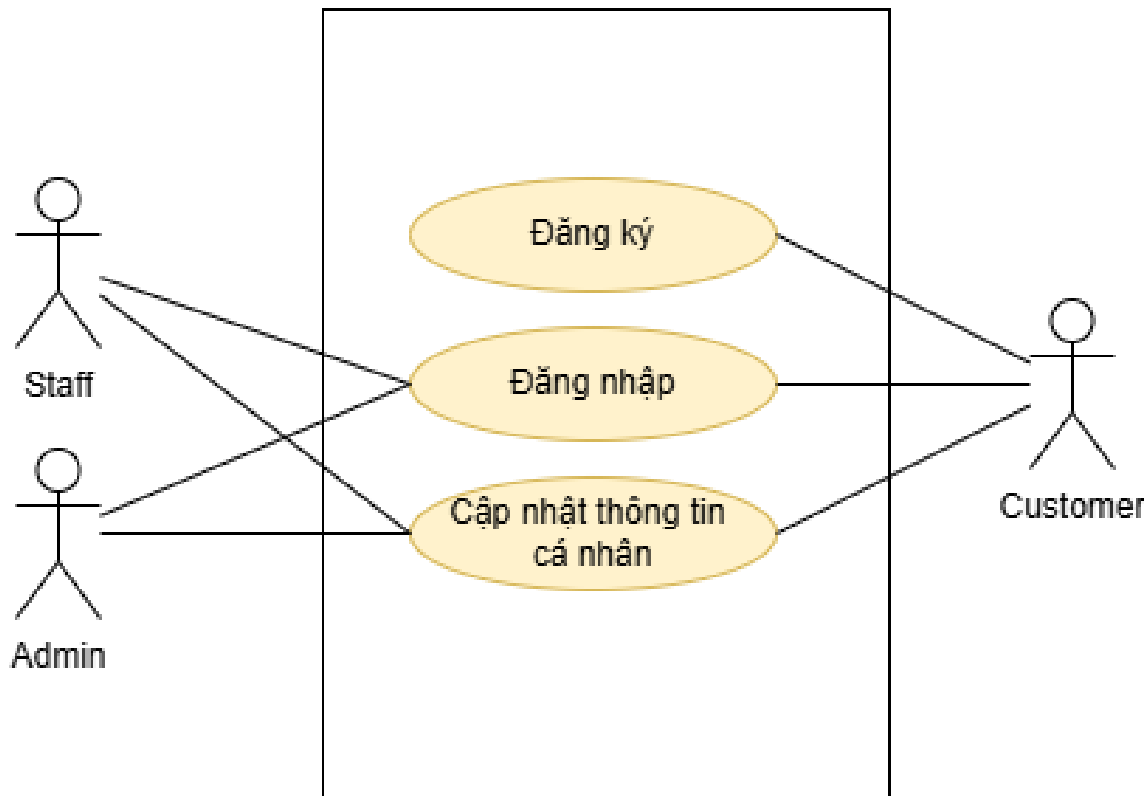
Biểu đồ use case phân rã Quản lý nhân viên được phân rã thành các use case với tác nhân là Admin như sau:

- **Xem danh sách nhân viên:** Cho phép Admin xem toàn bộ danh sách các nhân viên hiện có trong khách sạn.
- **Xem thông tin nhân viên:** Cho phép Admin xem thông tin chi tiết của một nhân viên cụ thể.
- **Thêm nhân viên:** Cho phép Admin tạo mới một tài khoản nhân viên và nhập

các thông tin cần thiết.

- **Xóa nhân viên:** Cho phép Admin loại bỏ tài khoản của một nhân viên khỏi hệ thống.

2.2.7 Biểu đồ use case phân rã Quản lý tài khoản

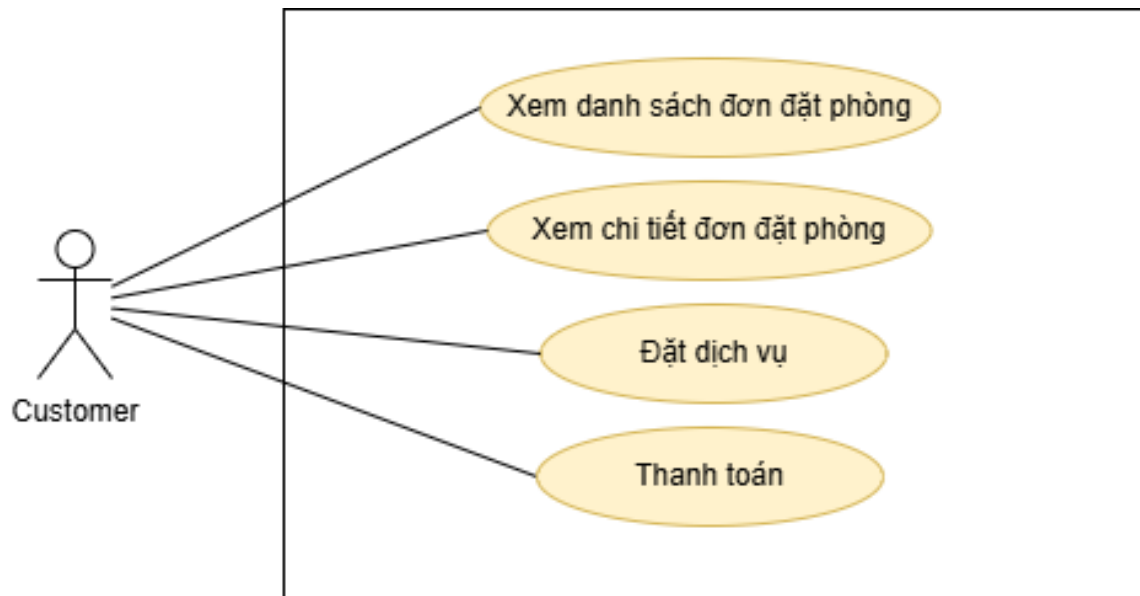


Hình 2.7: Biểu đồ use case phân rã Quản lý tài khoản

Biểu đồ use case phân rã Quản lý tài khoản được phân rã thành các use case với các tác nhân là Staff, Admin và Customer như sau:

- **Đăng ký:** Cho phép Customer tạo một tài khoản mới trên hệ thống để sử dụng các dịch vụ.
- **Đăng nhập:** Cho phép Staff, Admin và Customer truy cập vào hệ thống bằng tài khoản của mình.
- **Cập nhật thông tin cá nhân:** Cho phép Staff, Admin và Customer chỉnh sửa các thông tin cá nhân của mình đã lưu trong hệ thống.

2.2.8 Biểu đồ use case phân rã Quản lý đơn đặt



Hình 2.8: Biểu đồ use case phân rã Quản lý đơn đặt

Biểu đồ use case phân rã Quản lý đơn đặt được phân rã thành các use case với tác nhân là Customer như sau:

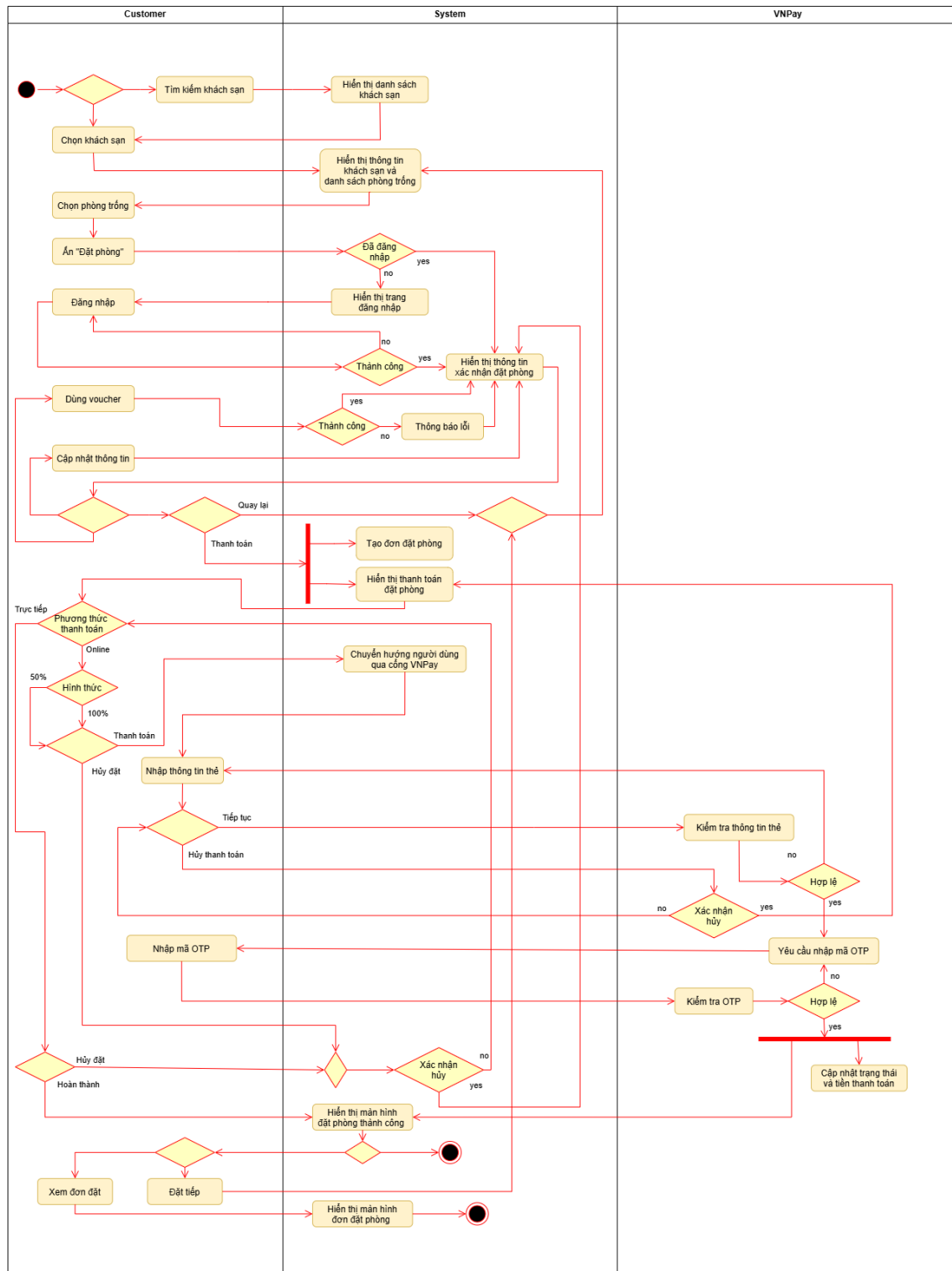
- **Xem danh sách đơn đặt phòng:** Cho phép Customer xem toàn bộ danh sách các đơn đặt phòng mà mình đã thực hiện.
- **Xem chi tiết đơn đặt phòng:** Cho phép Customer xem thông tin chi tiết của một đơn đặt phòng cụ thể.
- **Đặt dịch vụ:** Cho phép Customer yêu cầu hoặc đặt thêm các dịch vụ khác của khách sạn cho đơn đặt phòng hiện tại hoặc mới.
- **Thanh toán:** Cho phép Customer thực hiện quy trình thanh toán trực tuyến cho các đơn đặt phòng và dịch vụ đã sử dụng hoặc đặt.

2.2.9 Quy trình nghiệp vụ

Nghiệp vụ đặt phòng là một trong những chức năng trọng tâm của hệ thống quản lý và đặt phòng khách sạn. Quy trình này bao gồm việc khách hàng tìm kiếm khách sạn, lựa chọn thời gian lưu trú và loại phòng, sau đó cung cấp thông tin cá nhân cần thiết để tạo đơn đặt phòng.

Nghiệp vụ này không chỉ giúp khách hàng đảm bảo có chỗ ở trước khi đến khách sạn, mà còn hỗ trợ bộ phận lễ tân và quản lý trong việc kiểm soát tình trạng phòng, phân bổ tài nguyên hợp lý. Ngoài ra, nghiệp vụ đặt phòng còn có thể tích hợp với hệ thống khuyến mãi, voucher hoặc các dịch vụ bổ sung nhằm nâng cao trải nghiệm người dùng và tối đa hóa doanh thu cho khách sạn.

Dưới đây là biểu đồ hoạt động minh họa quá trình đặt phòng.



Hình 2.9: Biểu đồ hoan động quy trình nghiệp vụ Đặt phòng

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case Đặt phòng

Mã usecase	UC001	Tên usecase	Đặt phòng
Tác nhân	Customer (Khách hàng)		
Tiền điều kiện	Khách hàng đã truy cập trang chi tiết khách sạn.		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Customer	Chọn ngày nhận phòng, ngày trả phòng.
	2	System	Kiểm tra phòng trống và hiển thị các phòng phù hợp.
	3	Customer	Chọn phòng mong muốn và nhấn "Đặt phòng".
	4	System	Kiểm tra xem người dùng đã đăng nhập chưa.
	5	Customer	Xác nhận thông tin.
	6	Customer	(Tùy chọn) Nhập mã giảm giá (nếu có).
	7	System	Kiểm tra mã giảm giá và tính toán giá tiền.
	8	Customer	Nhấn nút "Thanh toán".
	9	System	Tạo đơn đặt phòng và hiển thị giao diện thanh toán.
	10	Customer	Chọn phương thức thanh toán.
	11	System	Chuyển hướng đến cổng thanh toán hoặc hiển thị thông tin thanh toán.
	12	Customer	Hoàn tất thanh toán.
	13	System	Cập nhật trạng thái phòng, trạng thái đơn đặt và hiển thị thông báo thành công.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	4a	System	Yêu cầu và chuyển đến trang đăng nhập nếu người dùng chưa đăng nhập.
	7a	System	Hệ thống báo lỗi nếu mã giảm giá không hợp lệ hoặc đã hết hạn.
	13a	System	Thanh toán không thành công. Hệ thống báo lỗi và cho phép thử lại hoặc chọn phương thức khác.
Hậu điều kiện	Không có		

Bảng 2.2: Đặc tả use case Đặt phòng

2.3.2 Đặc tả use case Xem dự đoán lượng khách

Mã usecase	UC002	Tên usecase	Xem dự đoán lượng khách
Tác nhân	Admin (Quản lý khách sạn)		
Tiền điều kiện	Người dùng đã đăng nhập với quyền quản lý.		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Admin	Truy cập trang dự đoán lượng khách.
	2	System	Chuẩn bị các dữ liệu cần thiết để tiết hành dự đoán.
	3	System	Tiến hành dự đoán bằng các mô hình học máy.
	4	System	Lưu kết quả dự đoán vào cơ sở dữ liệu và hiện thị trang dự đoán.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	1a	System	Nếu chưa đăng nhập hoặc không đủ quyền, chuyển hướng đến trang đăng nhập hoặc báo lỗi truy cập.
Hậu điều kiện	Biểu đồ được hiển thị.		

Bảng 2.3: Đặc tả use case Xem dự đoán lượng khách

2.3.3 Đặc tả use case Thêm đơn đặt phòng

Mã usecase	UC003	Tên usecase	Thêm đơn đặt phòng
Tác nhân	Staff (Nhân viên lễ tân)		
Tiền điều kiện	Người dùng đã đăng nhập với quyền nhân viên.		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Staff	Truy cập trang quản lý phòng.
	2	Staff	Chọn loại phòng và ấn nút biểu tượng hình chiếc giường.
	3	System	Hiện thị danh sách các phòng tương ứng của loại phòng đã chọn.
	4	Staff	Chọn phòng và ấn nút biểu tượng hình hóa đơn.
	5	System	Hiện thị hộp thoại tạo đơn đặt phòng.
	6	Staff	Nhập số điện thoại hoặc tên tài khoản khách hàng.
	7	System	Kiểm tra và hiển thị thông tin khách hàng.
	8	System	Tạo đơn đặt phòng và lưu vào cơ sở dữ liệu.
	9	System	Thông báo tạo đơn đặt phòng thành công.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	7a	System	Nếu khách hàng chưa có tài khoản, hiển thị nút tạo tài khoản.
	7b	Staff	Nhấn nút "tạo tài khoản".
	7c	System	Tạo tài khoản khách hàng và hiển thị thông tin khách hàng.
Hậu điều kiện	Đơn đặt phòng được lưu thành công và sẵn sàng cho các bước tiếp theo (thanh toán, xác nhận,...).		

Bảng 2.4: Đặc tả use case Thêm đơn đặt phòng

2.3.4 Đặc tả use case Thêm mã giảm giá

Mã usecase	UC004	Tên usecase	Thêm mã giảm giá
Tác nhân	Admin (Quản lý khách sạn)		
Tiền điều kiện	Người dùng đã đăng nhập với quyền admin.		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Admin	Truy cập trang quản lý voucher.
	2	Admin	Chọn chức năng thêm voucher mới.
	3	Admin	Nhập thông tin mã giảm giá (mã, phần trăm, hạn dùng, loại giảm giá,...).
	4	System	Kiểm tra hợp lệ và lưu mã giảm giá vào hệ thống.
	5	System	Hiển thị thông báo thêm mã thành công.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	4a	System	Nếu thông tin không hợp lệ (ví dụ: trùng mã, định dạng sai), hiển thị thông báo lỗi và yêu cầu nhập lại.
Hậu điều kiện	Mã giảm giá mới được thêm vào hệ thống và có thể áp dụng khi đặt phòng.		

Bảng 2.5: Đặc tả use case Thêm mã giảm giá

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng cốt lõi, để đảm bảo hệ thống vận hành hiệu quả và mang lại trải nghiệm tối ưu cho người dùng, hệ thống cần đáp ứng thêm các yêu cầu phi chức năng quan trọng như hiệu năng, tính tin cậy, tính dễ sử dụng, tính dễ bảo trì và tính bảo mật.

Về hiệu năng, hệ thống cần đảm bảo tốc độ phản hồi nhanh, thời gian tải trang không vượt quá 5 giây ngay cả trong giờ cao điểm có nhiều người dùng đồng thời truy cập và đặt phòng.

Về tính tin cậy, hệ thống phải hoạt động ổn định 24/7, đảm bảo dữ liệu đơn đặt phòng, thanh toán và thông tin khách hàng không bị mất hoặc sai lệch trong quá trình xử lý.

Về tính dễ sử dụng, giao diện người dùng (UI) cần được thiết kế trực quan, thân thiện với người dùng phổ thông. Khách hàng không cần hướng dẫn vẫn có thể đặt phòng dễ dàng.

Về tính dễ bảo trì, mã nguồn và kiến trúc hệ thống cần được tổ chức rõ ràng theo chuẩn MVC (Model-View-Controller), dễ dàng mở rộng và sửa lỗi. Hệ thống cần có tài liệu hướng dẫn triển khai và bảo trì cho đội ngũ kỹ thuật.

Về tính bảo mật, hệ thống cần bảo mật thông tin cá nhân và có xác thực người dùng bằng JWT.

2.5 Tổng kết

Tổng kết lại, chương này đã cung cấp cái nhìn toàn diện về bối cảnh thực tế và nhu cầu người dùng đối với hệ thống quản lý và đặt phòng khách sạn. Thông qua việc khảo sát các hệ thống tương tự, phân tích yêu cầu từ hai nhóm đối tượng chính và đánh giá xu hướng công nghệ, chương đã xác định rõ các chức năng trọng tâm cần triển khai. Bên cạnh đó, các yêu cầu chức năng và phi chức năng cũng đã được đặc tả cụ thể, làm nền tảng định hướng cho quá trình thiết kế kiến trúc hệ thống, triển khai giải pháp kỹ thuật và kiểm thử phần mềm trong các chương tiếp theo.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Trong chương này, đồ án trình bày tổng quan, ưu điểm, nhược điểm và ứng dụng của các công nghệ, nền tảng và công cụ được sử dụng để xây dựng hệ thống quản lý và đặt phòng khách sạn tích hợp dự đoán lượng khách hàng. Các công nghệ này được lựa chọn dựa trên yêu cầu thực tế đã phân tích ở Chương 2, bao gồm cả yêu cầu chức năng lẫn phi chức năng, đồng thời cân nhắc đến tính khả thi, hiệu quả và mức độ phù hợp với quy mô của hệ thống.

3.1 React.js

React là một thư viện JavaScript mã nguồn mở do Facebook phát triển, được sử dụng để xây dựng giao diện người dùng dựa trên các component có thể tái sử dụng.

3.1.1 Ưu điểm

- Virtual DOM giúp cập nhật giao diện nhanh và hiệu quả.
- Thiết kế dựa trên component giúp dễ quản lý và mở rộng.
- Tài nguyên học tập và thư viện hỗ trợ phong phú.
- Hỗ trợ server-side rendering và cải thiện khả năng SEO khi cần thiết.

3.1.2 Nhược điểm

- Chỉ tập trung vào phần *View*, nên cần tích hợp thêm thư viện như Redux hoặc Zustand để quản lý trạng thái phức tạp.
- JSX và các khái niệm như *hook*, *context* có thể gây khó hiểu đối với người mới bắt đầu.

3.1.3 Ứng dụng trong đồ án

React được sử dụng để xây dựng giao diện người dùng, mang lại trải nghiệm mượt mà và thân thiện cho cả khách hàng và nhân viên khách sạn khi tương tác với hệ thống.

3.1.4 Lý do lựa chọn

Trong quá trình đánh giá các thư viện và framework frontend, em đã cân nhắc một số lựa chọn phổ biến:

- **Vue.js:** Dễ học, nhẹ, thích hợp với dự án nhỏ hoặc vừa. Tuy nhiên, tài nguyên học tập còn hạn chế hơn so với React.
- **Angular:** Cung cấp giải pháp toàn diện từ đầu đến cuối (full-stack frontend framework), nhưng lại phức tạp, có thể thừa tính năng và gây nặng nề không cần thiết cho một ứng dụng vừa như hệ thống đặt phòng.

Từ đó, em lựa chọn **React** vì tính linh hoạt, phổ biến rộng rãi trong cộng đồng, dễ học đối với người đã quen JavaScript và dễ dàng tích hợp với hệ thống REST API backend.

3.2 Node.js & Express.js

Node.js là nền tảng máy chủ mã nguồn mở sử dụng JavaScript, vận hành trên mô hình bất đồng bộ, thích hợp để xây dựng các ứng dụng mạng có hiệu suất cao. Express.js là framework phát triển web chạy trên Node.js, cung cấp bộ công cụ mạnh mẽ để xây dựng các API hoặc ứng dụng web nhẹ.

3.2.1 Ưu điểm

- Xử lý song song nhiều yêu cầu, phù hợp với hệ thống đặt phòng có nhiều người dùng truy cập cùng lúc.
- Tài nguyên phong phú, có hàng nghìn thư viện hỗ trợ tích hợp nhanh chóng.
- Dễ triển khai REST API, hỗ trợ tốt cho mô hình frontend-backend tách biệt.
- Được cộng đồng lớn hỗ trợ, có nhiều tài nguyên học tập.

3.2.2 Nhược điểm

- Quản lý callback hoặc promise phức tạp nếu không sử dụng đúng cách (dễ xảy ra *callback hell*).
- Không phù hợp với các tác vụ cần tính toán nặng do chỉ sử dụng một luồng chính.

3.2.3 Ứng dụng trong đồ án

Node.js và Express.js được sử dụng để xây dựng backend của hệ thống, xử lý logic nghiệp vụ và cung cấp các API cho frontend sử dụng. Công nghệ này giúp đảm bảo hệ thống phản hồi nhanh, dễ mở rộng và duy trì.

3.2.4 Lý do lựa chọn

Trong quá trình khảo sát công nghệ backend, em đã xem xét một số lựa chọn phổ biến:

- **Django (Python):** Có khả năng tích hợp mạnh, bảo mật cao, phù hợp với các ứng dụng lớn và yêu cầu phức tạp. Tuy nhiên, framework này khá nặng và khó học tập nếu đã quen với JavaScript.
- **Spring Boot (Java):** Cho hiệu suất cao, mạnh mẽ trong môi trường doanh nghiệp. Tuy nhiên, cấu hình ban đầu phức tạp, thời gian triển khai dài, không phù hợp với việc phát triển nhỏ hoặc thời gian hạn chế.

Từ đó, em lựa chọn **Node.js kết hợp với Express.js** vì sự đơn giản, dễ học, tốc

độ phát triển nhanh và khả năng tích hợp tốt với JavaScript — đặc biệt phù hợp với ứng dụng web dạng CRUD như hệ thống đặt phòng khách sạn.

3.3 MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở phổ biến, sử dụng ngôn ngữ truy vấn SQL để quản lý dữ liệu. Nó thường được sử dụng trong các hệ thống yêu cầu lưu trữ dữ liệu có quan hệ rõ ràng như khách sạn, nhân viên, đặt phòng.

3.3.1 Ưu điểm

- Hiệu suất ổn định với các thao tác CRUD.
- Tích hợp tốt với các ORM như Sequelize.
- Tài liệu đầy đủ, dễ học và vận hành.
- Cộng đồng hỗ trợ lớn, có nhiều công cụ quản lý cơ sở dữ liệu.

3.3.2 Nhược điểm

- Hạn chế trong xử lý dữ liệu phi cấu trúc.
- Một số tính năng nâng cao bị giới hạn so với PostgreSQL (ví dụ như full-text search, JSON).

3.3.3 Ứng dụng trong đồ án

MySQL được sử dụng để lưu trữ thông tin về khách sạn, người dùng, phòng, đơn đặt phòng, thanh toán,... phù hợp với bài toán quản lý dữ liệu có cấu trúc rõ ràng.

3.3.4 Lý do lựa chọn

Một số lựa chọn phổ biến mà em đã từng xem xét và cân nhắc khác như:

- **PostgreSQL**: Mạnh hơn về tính năng, nhưng phức tạp hơn khi triển khai và tối ưu hiệu năng.
- **MongoDB**: Phù hợp với dữ liệu phi cấu trúc, nhưng không tối ưu với các quan hệ rõ ràng.

Vì dữ liệu của hệ thống là dạng bảng, có quan hệ rõ ràng và nhiều phép *join*, nên **MySQL** là lựa chọn phù hợp và tối ưu.

3.4 Sequelize ORM

Sequelize là một ORM (Object-Relational Mapping) dành cho Node.js, giúp ánh xạ dữ liệu giữa các đối tượng JavaScript và bảng dữ liệu trong hệ quản trị cơ sở dữ liệu như MySQL.

3.4.1 Ưu điểm

- Viết truy vấn đơn giản hơn, giảm thiểu lỗi so với viết câu lệnh SQL thủ công.
- Hỗ trợ định nghĩa quan hệ giữa các bảng như quan hệ 1-n, n-n.
- Có tính năng migration giúp theo dõi và quản lý thay đổi cấu trúc cơ sở dữ liệu.

3.4.2 Nhược điểm

- Hiệu suất có thể thấp hơn khi thực hiện các truy vấn phức tạp.
- Cần học cú pháp riêng biệt của ORM, việc debug trong một số trường hợp có thể khó khăn.

3.4.3 Ứng dụng trong đồ án

Sequelize được sử dụng để giúp tổ chức mã nguồn rõ ràng, dễ mở rộng và bảo trì khi hệ thống phát triển.

3.4.4 Lý do lựa chọn

- **Prisma**: Cú pháp rõ ràng, dễ đọc nhưng cộng đồng nhỏ hơn.
- **TypeORM**: Tích hợp tốt với TypeScript, nhưng cấu hình phức tạp hơn.

Sequelize được lựa chọn vì là một ORM phổ biến, có tài liệu đầy đủ, hỗ trợ tốt cho **MySQL** và dễ tích hợp với Express.js trong hệ sinh thái JavaScript.

3.5 Flask (Python)

Flask là một *micro-framework* viết bằng ngôn ngữ Python, phù hợp để xây dựng các dịch vụ web nhỏ gọn, đơn giản. Flask thường được sử dụng để triển khai các mô hình học máy và cung cấp các API phục vụ hệ thống chính.

3.5.1 Ưu điểm

- Nhẹ, cấu hình tối giản, dễ triển khai và chạy độc lập.
- Tích hợp tốt với các thư viện học máy như *scikit-learn*, *pandas*.
- Hỗ trợ xây dựng API rõ ràng, thích hợp với các dịch vụ nội bộ gọi từ backend chính.

3.5.2 Nhược điểm

- Không phù hợp cho các ứng dụng lớn nếu không tổ chức cấu trúc dự án rõ ràng.
- Khi hệ thống phức tạp, cần tự quản lý nhiều thành phần thủ công như authentication, logging, ...

3.5.3 Ứng dụng trong đồ án

Flask được sử dụng để triển khai mô hình học máy dự đoán lượng khách lưu trú theo từng ngày. Hệ thống Flask hoạt động như một dịch vụ độc lập và được kết nối với backend chính viết bằng Node.js thông qua các API REST.

3.5.4 Lý do lựa chọn

- **FastAPI:** Hiện đại hơn, hỗ trợ xử lý bất đồng bộ tốt nhưng chưa quen thuộc với nhóm.
- **Django REST Framework:** Toàn diện nhưng khá nặng nề cho dự án nhỏ.

Flask được lựa chọn vì gọn nhẹ, dễ tích hợp và triển khai nhanh chóng các mô hình học máy đơn giản như Linear Regression.

3.6 Random Forest

Random Forest là một thuật toán học máy thuộc nhóm *ensemble learning*, hoạt động bằng cách kết hợp nhiều cây quyết định (*decision trees*) để đưa ra kết quả dự đoán chính xác và ổn định hơn. Mỗi cây trong rừng được huấn luyện trên một tập con ngẫu nhiên của tập dữ liệu gốc.

3.6.1 Ưu điểm

- Hiệu suất cao khi xử lý dữ liệu phi tuyến tính hoặc có nhiều đặc trưng phức tạp.
- Khả năng chống *overfitting* tốt nhờ cơ chế tổng hợp dự đoán từ nhiều cây.
- Không yêu cầu chuẩn hóa dữ liệu đầu vào như các thuật toán tuyến tính.

3.6.2 Nhược điểm

- Mô hình lớn và phức tạp nên quá trình huấn luyện và suy luận có thể chậm với dữ liệu lớn.
- Kết quả khó giải thích hơn so với các mô hình đơn giản như hồi quy tuyến tính.

3.6.3 Ứng dụng trong đồ án

Random Forest được sử dụng để dự đoán số lượng khách đặt phòng theo ngày, dựa trên các yếu tố như thời điểm, loại phòng, vị trí khách sạn, ngày trong tuần, v.v. Mô hình này hỗ trợ hệ thống đưa ra các dự đoán chính xác nhằm phục vụ mục tiêu tối ưu hóa vận hành và tài nguyên.

3.6.4 Lý do lựa chọn

Mặc dù không phải là mô hình đơn giản nhất nhưng **Random Forest** dễ triển khai, hiệu quả với dữ liệu có quy mô vừa và nhiều chiều, đồng thời không yêu cầu

quá nhiều bước tinh chỉnh hoặc xử lý dữ liệu đầu vào phức tạp.

3.7 XGBoost

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy thuộc nhóm *boosting*, được phát triển nhằm tối ưu hóa hiệu suất huấn luyện thông qua kỹ thuật *gradient boosting*. Đây là một trong những thuật toán phổ biến và đạt kết quả cao trong các cuộc thi học máy nhờ khả năng tổng quát hóa và hiệu năng vượt trội.

3.7.1 Ưu điểm

- Độ chính xác cao trong nhiều bài toán thực tế.
- Có thể xử lý tốt dữ liệu thiếu hoặc không cần quá nhiều bước tiền xử lý.
- Hỗ trợ song song hóa, tối ưu hiệu suất trên tập dữ liệu lớn.

3.7.2 Nhược điểm

- Phức tạp trong việc tinh chỉnh các *hyperparameter*.
- Thời gian huấn luyện có thể lâu hơn so với các mô hình đơn giản.

3.7.3 Ứng dụng trong đồ án

XGBoost được sử dụng trong bài toán dự đoán lượng khách lưu trú trong tương lai. Với khối lượng dữ liệu huấn luyện lớn và đa chiều, mô hình XGBoost giúp cải thiện độ chính xác dự đoán so với các thuật toán truyền thống như hồi quy tuyến tính hay cây quyết định đơn lẻ.

3.7.4 Lý do lựa chọn

XGBoost mang lại sự cân bằng tốt giữa độ chính xác và hiệu năng xử lý, là lựa chọn phù hợp với bài toán dự báo định lượng có độ phức tạp trung bình đến cao.

3.8 SARIMA

SARIMA (Seasonal Autoregressive Integrated Moving Average) là một mô hình thống kê mở rộng từ ARIMA, được thiết kế để xử lý các chuỗi thời gian có tính mùa vụ rõ rệt. Đây là một công cụ hữu ích trong việc phân tích xu hướng và dự báo dữ liệu theo chu kỳ.

3.8.1 Ưu điểm

- Phù hợp với dữ liệu chuỗi thời gian có tính chu kỳ như theo tuần, tháng.
- Có khả năng giải thích rõ ràng cấu trúc thời gian của dữ liệu.
- Không yêu cầu nhiều biến độc lập như các mô hình học máy.

3.8.2 Nhược điểm

- Cần dữ liệu ổn định và đủ dài để mô hình hoạt động hiệu quả.

- Phức tạp trong việc lựa chọn và tinh chỉnh các tham số: (p, d, q, P, D, Q, s) .
- Hạn chế trong việc xử lý các yếu tố không mang tính thời gian.

3.8.3 Ứng dụng trong đồ án

SARIMA được áp dụng để phân tích và dự báo số lượng khách theo các chu kỳ cố định như ngày trong tuần hoặc mùa trong năm. Việc này hỗ trợ hệ thống xác định xu hướng và giúp quản lý đưa ra kế hoạch kinh doanh phù hợp với từng giai đoạn.

3.8.4 Lý do lựa chọn

Sau khi phân tích và nghiên cứu một số mô hình tương tự như:

- **ARIMA:** Không xử lý được tính mùa vụ, chỉ phù hợp với chuỗi không có yếu tố theo mùa.
- **LSTM:** Là mô hình học sâu mạnh mẽ cho chuỗi thời gian nhưng đòi hỏi nhiều tài nguyên tính toán và dữ liệu lớn.

Em nhận thấy dữ liệu khách lưu trú có tính mùa vụ rõ ràng, và trong bối cảnh không có nhiều biến độc lập, **SARIMA** là lựa chọn phù hợp nhờ khả năng giải thích tốt và dễ triển khai so với các mô hình học sâu.

3.9 Tổng kết

Trong chương này, các công nghệ và mô hình chính được lựa chọn cho hệ thống đã được trình bày và phân tích. Ở phía frontend, React.js được chọn nhờ khả năng xây dựng giao diện hiện đại, dễ mở rộng và mang lại trải nghiệm mượt mà cho người dùng. Backend sử dụng Express.js kết hợp MySQL làm cơ sở dữ liệu quan hệ, cùng Sequelize ORM để quản lý dữ liệu hiệu quả và dễ bảo trì. Đối với phần dự đoán, mô hình học máy được triển khai riêng biệt bằng Flask, đảm bảo tính linh hoạt và dễ tích hợp.

Ba mô hình được sử dụng gồm Random Forest, XGBoost và SARIMA, mỗi mô hình phù hợp với những đặc điểm dữ liệu khác nhau, giúp nâng cao độ chính xác trong dự báo lượng khách lưu trú. Các công nghệ và mô hình này đã được lựa chọn nhằm đảm bảo hiệu năng, độ chính xác và khả năng mở rộng của hệ thống.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Khép lại chương 3 với phần giới thiệu về các công nghệ và các mô hình học máy được sử dụng, chương 4 sẽ trình bày toàn bộ quá trình thiết kế và hiện thực hóa hệ thống quản lý và đặt phòng khách sạn từ kiến trúc tổng thể đến triển khai chi tiết. Mở đầu là việc lựa chọn và áp dụng kiến trúc phần mềm phù hợp nhằm đảm bảo tính phân tầng, khả năng mở rộng và bảo trì. Tiếp theo, chương trình bày các mô hình thiết kế UML như biểu đồ gói, biểu đồ lớp và biểu đồ trình tự, làm rõ cấu trúc hệ thống và mối quan hệ giữa các thành phần.

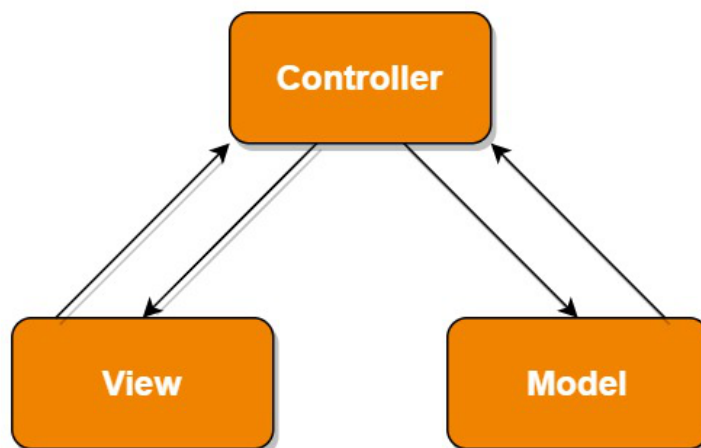
Phần tiếp theo mô tả quá trình xây dựng giao diện người dùng, thiết kế cơ sở dữ liệu, xây dựng các module học máy và phát triển chức năng thông qua các công cụ và công nghệ đã lựa chọn. Cuối cùng, chương đi sâu vào kiểm thử và triển khai thực tế hệ thống, nhằm đánh giá hiệu năng, độ ổn định và mức độ đáp ứng yêu cầu người dùng.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

a, Lựa chọn kiến trúc: MVC (Model – View – Controller)

Trong đồ án này, kiến trúc MVC (Model – View – Controller) được lựa chọn để xây dựng hệ thống quản lý và đặt phòng khách sạn. Đây là kiến trúc phần mềm phổ biến và phù hợp với các ứng dụng web hiện đại có sự phân tách rõ giữa giao diện người dùng, xử lý nghiệp vụ và quản lý dữ liệu.



Hình 4.1: mô hình MVC

b, Model (M) – Xử lý dữ liệu và nghiệp vụ ứng dụng

Model là nơi quản lý dữ liệu, thực hiện các thao tác đọc/ghi vào cơ sở dữ liệu, đồng thời chịu trách nhiệm hiện thực các quy tắc nghiệp vụ cốt lõi của hệ thống. Model đóng vai trò như “trái tim” của ứng dụng khi nó lưu trữ trạng thái, các cấu trúc dữ liệu, cũng như đảm bảo tính toàn vẹn dữ liệu và tuân thủ các ràng buộc nghiệp vụ.

Trong hệ thống quản lý khách sạn, Model đảm nhiệm:

- Định nghĩa các bảng dữ liệu như người dùng, khách sạn, phòng, đơn đặt phòng, mã giảm giá,...
- Kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi ghi vào cơ sở dữ liệu.
- Triển khai các hàm nghiệp vụ như: kiểm tra phòng còn trống, áp dụng voucher, tính tổng tiền thanh toán, cập nhật trạng thái đặt phòng,...
- Giao tiếp với cơ sở dữ liệu thông qua ORM (Sequelize), giúp giảm độ phức tạp và nâng cao hiệu suất xử lý.

c, View (V) – Giao diện người dùng

View chịu trách nhiệm hiển thị dữ liệu và cung cấp giao diện tương tác giữa hệ thống và người dùng. View không xử lý logic nghiệp vụ mà chỉ phản ánh kết quả từ Controller gửi về, đồng thời thu thập các thao tác (click, nhập liệu, chọn lựa,...) của người dùng để gửi yêu cầu tiếp theo.

Trong đồ án này, View được hiện thực bằng React.js và đảm nhiệm:

- Thiết kế các trang như: Trang chủ, Đặt phòng, Đăng nhập, Quản lý đặt phòng, Quản trị khách sạn,...
- Thể hiện dữ liệu như danh sách phòng, thông tin đặt phòng, biểu đồ doanh thu,...
- Tạo trải nghiệm người dùng mượt mà bằng các kỹ thuật như routing động, hiển thị loading khi đang xử lý,...
- Giao tiếp với Controller thông qua API gọi từ frontend đến backend bằng phương thức HTTP.

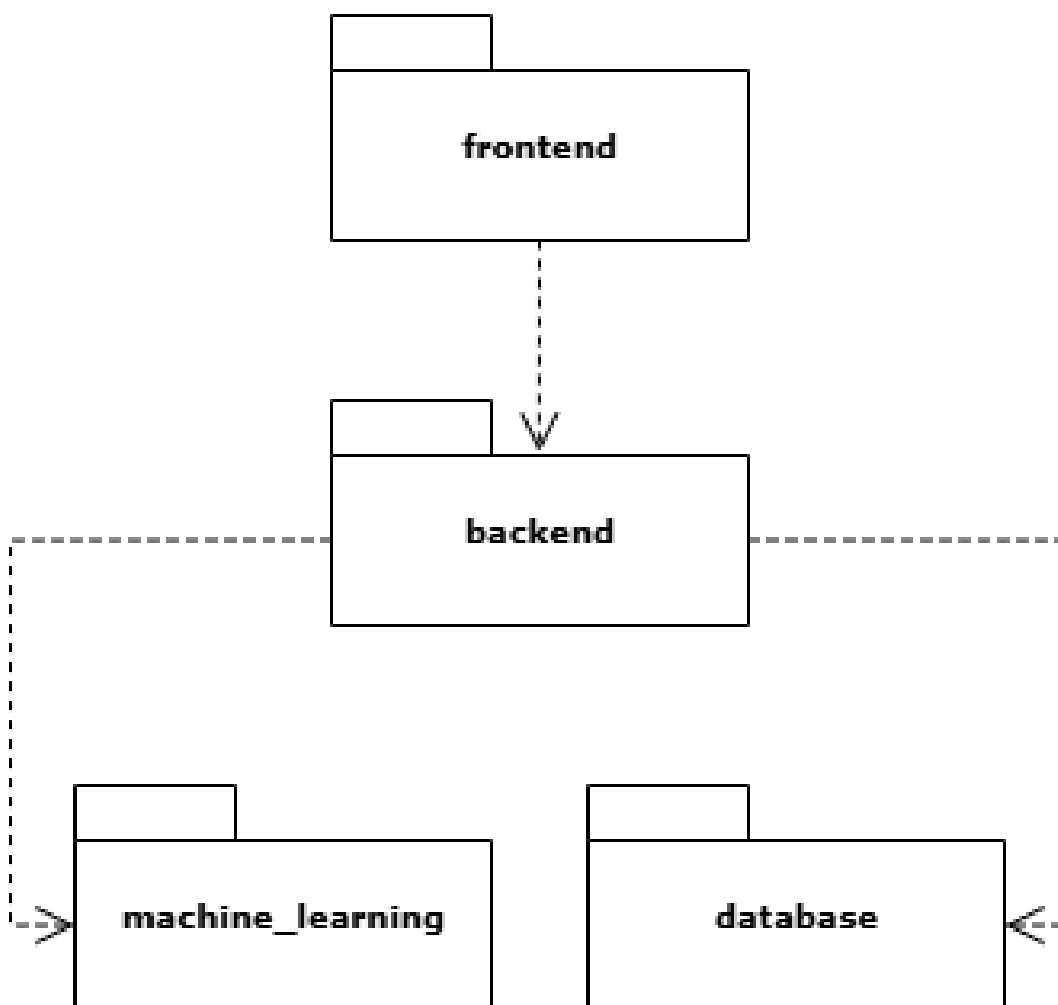
d, Controller (C) – Cầu nối xử lý giữa người dùng và hệ thống

Controller đóng vai trò là trung gian giữa View và Model. Nó nhận yêu cầu từ giao diện người dùng (View), xử lý logic cần thiết hoặc kiểm tra quyền truy cập, rồi tương tác với Model để truy xuất dữ liệu hoặc thay đổi trạng thái hệ thống. Sau đó, Controller phản hồi lại View với kết quả tương ứng.

Trong hệ thống của đồ án:

- Controller được xây dựng bằng Express.js.
- Các route RESTful (ví dụ: /api/bookings, /api/rooms) xử lý các thao tác như đặt phòng, đăng ký, đăng nhập, cập nhật trạng thái,...
- Kết hợp middleware như `authMiddleware` để tăng tính bảo mật và đảm bảo tính toàn vẹn yêu cầu.
- Gọi các service để truy xuất dữ liệu, rồi trả dữ liệu về dưới dạng JSON cho frontend xử lý hiển thị.

4.1.2 Thiết kế tổng quan



Hình 4.2: Biểu đồ gói tổng quan

a, Gói frontend

Mục đích: Đây là gói chịu trách nhiệm chính cho việc hiển thị giao diện người dùng và tiếp nhận các tương tác từ người dùng như khách đặt phòng, xem thông tin khách sạn, quản trị viên cập nhật dữ liệu,... Gói này đóng vai trò là cửa ngõ đầu

tiên tiếp xúc với người dùng trong hệ thống.

Nhiệm vụ:

- Hiển thị dữ liệu được lấy từ backend một cách trực quan, dễ hiểu.
- Đảm bảo trải nghiệm người dùng nhất quán, thân thiện và dễ sử dụng.
- Gửi các yêu cầu (API call) về backend để xử lý: như đăng nhập, đặt phòng, xem biểu đồ dự báo,...
- Giao tiếp một chiều với backend, không phụ thuộc trực tiếp vào machine learning hay database.

b, Gói backend

Mục đích: Đây là gói trung tâm xử lý logic nghiệp vụ, đóng vai trò là cầu nối giữa giao diện người dùng và các tài nguyên hệ thống như cơ sở dữ liệu hoặc các mô hình học máy. Toàn bộ các thao tác người dùng gửi từ frontend đều được tiếp nhận và xử lý tại đây.

Nhiệm vụ:

- Xử lý các luồng nghiệp vụ: xác thực người dùng, đặt phòng, quản lý khách sạn, sử dụng voucher,...
- Giao tiếp với database để đọc/ghi dữ liệu bằng ORM Sequelize.
- Gửi yêu cầu đến `machine_learning` để lấy kết quả dự đoán lượng khách.
- Bảo đảm tính bảo mật, xác thực và phân quyền.
- Giữ cho frontend không cần biết cách xử lý nội bộ hoặc cấu trúc dữ liệu.

c, Gói database

Mục đích: Đây là gói quản lý dữ liệu bền vững cho toàn hệ thống, đảm bảo lưu trữ an toàn và truy xuất hiệu quả cho các thành phần khác. Gói này là “xương sống dữ liệu” của toàn bộ ứng dụng.

Nhiệm vụ:

- Lưu trữ thông tin khách sạn, người dùng, loại phòng, đặt phòng, trạng thái thanh toán, voucher,...
- Đảm bảo tính nhất quán, toàn vẹn dữ liệu trong hệ thống.
- Cung cấp các API thao tác dữ liệu thông qua backend (sử dụng ORM Sequelize).
- Tách biệt với frontend, chỉ giao tiếp gián tiếp thông qua backend.

d, Gói `machine_learning`

Mục đích: Đây là gói chứa các mô hình học máy giúp dự đoán lượng khách hàng ngày, nhằm hỗ trợ nhà quản lý khách sạn ra quyết định chính xác hơn về nhân lực, tài nguyên và chiến lược kinh doanh.

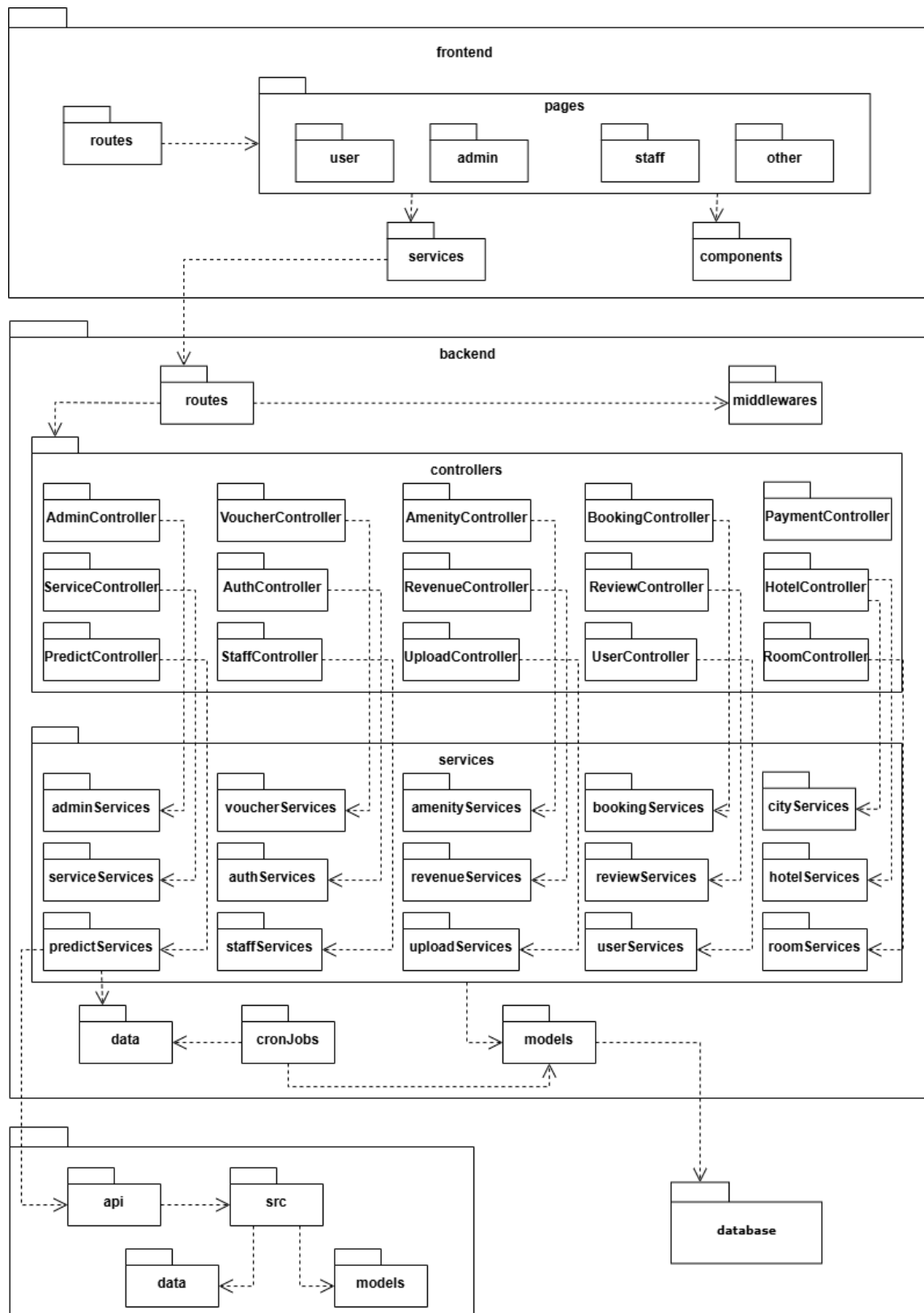
Nhiệm vụ:

- Triển khai các mô hình học máy như Random Forest, XGBoost và SARIMA.
- Nhận yêu cầu dự đoán từ backend qua API Flask, xử lý và trả lại kết quả dự báo.
- Không phụ thuộc ngược vào backend hay frontend.
- Gói này có thể mở rộng độc lập, dễ bảo trì và thay thế mô hình.

e, Mối quan hệ giữa các gói

- **frontend** → **backend**: Gói `frontend` phụ thuộc vào `backend` để gửi và nhận dữ liệu thông qua các API. Tất cả các chức năng như đăng nhập, đặt phòng, thanh toán, dự đoán,... đều được gửi về `backend` để xử lý và trả kết quả hiển thị.
- **backend** → **database**: Gói `backend` phụ thuộc vào `database` để thực hiện các thao tác lưu trữ và truy xuất dữ liệu như thông tin khách sạn, phòng, người dùng, đơn đặt phòng, giao dịch,... Tương tác này được thực hiện thông qua ORM `Sequelize`.
- **backend** → **machine_learning**: Gói `backend` phụ thuộc vào `machine_learning` để gọi các mô hình học máy thực hiện dự đoán lượng khách theo ngày, tuần hoặc tháng. Giao tiếp này được thực hiện qua các REST API nội bộ.

4.1.3 Thiết kế chi tiết gói



Hình 4.3: Biểu đồ gói chi tiết

Hệ thống được chia thành hai phần chính: **frontend** và **backend**, tuân thủ kiến trúc phân tầng rõ ràng theo mô hình **MVC**. Ngoài ra, phần **machine_learning** được

triển khai riêng như một dịch vụ độc lập.

a, Frontend

- `routes`: Xác định các tuyến đường (route) cho người dùng và điều hướng đến các trang tương ứng.
- `pages`: Chứa các giao diện phân theo vai trò (user, admin, staff) và các trang khác.
- `services`: Tầng trung gian gọi API từ backend, đảm nhiệm việc gửi và nhận dữ liệu.
- `components`: Các thành phần UI tái sử dụng như button, input, v.v.

b, Backend

- `routes`: Định tuyến API đến các controller phù hợp.
- `controllers`: Xử lý logic nhận request, gọi service tương ứng, và trả response.
- `services`: Tầng nghiệp vụ, xử lý logic chính và tương tác với cơ sở dữ liệu.
- `middlewares`: Quản lý xác thực, phân quyền.
- `models`: Định nghĩa cấu trúc bảng dữ liệu sử dụng ORM Sequelize.
- `data`: Lưu trữ các tập dữ liệu để huấn luyện mô hình.
- `cronJob`: Tự động chạy các tác vụ theo lịch như hủy đơn chưa thanh toán sau 30 phút, cập nhật dữ liệu để tái huấn luyện mô hình v.v.

c, Machine_Learning

- `src`: Chứa các hàm chức năng như xử lý dữ liệu, huấn luyện mô hình.
- `data`: Lưu trữ dữ liệu được gửi đến và dữ liệu đã được xử lý.
- `models`: Lưu trữ các mô hình sau khi được huấn luyện.
- `api`: Chứa các api như dự đoán lượng khách.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

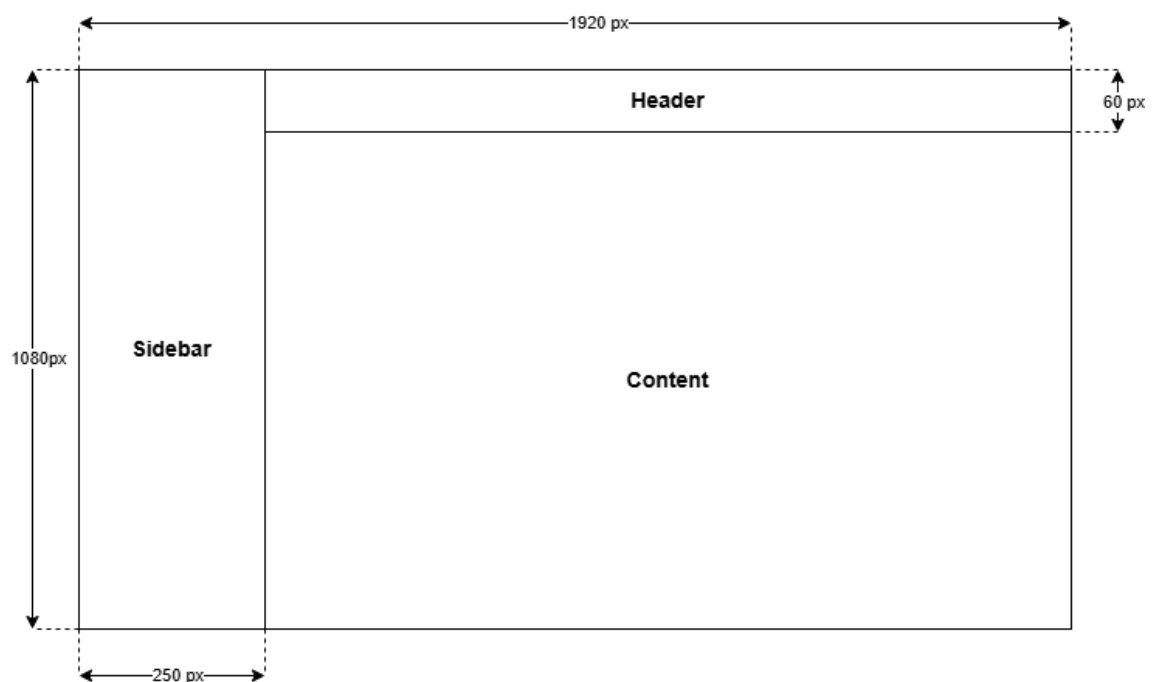
a, Thông số kỹ thuật

Dưới đây là bảng các thông số kỹ thuật màn hình được áp dụng trong đồ án.

STT	Thông tin	Chi tiết
1	Độ phân giải màn hình	Độ phân giải tốt nhất được hỗ trợ là 1920 x 1080
2	Tỷ lệ khung hình	16:9
3	Kích thước màn hình	Từ 13 inch đến 2 inch
4	Số lượng màu sắc hỗ trợ	Toàn bộ màu trong bảng RGB
5	Tương thích trình duyệt	Chrome, Edge, Firefox, Safari
6	Nút bấm	Chiều cao: 40px Cỡ chữ: 14px
7	Màu sắc chủ đạo	#000000, #ffffff, #0194f3
8	Font chữ	Open Sans
9	Vị trí hiển thị thông điệp phản hồi	Phía trên bên phải

Bảng 4.1: Thông số kỹ thuật màn hình

b, Bố cục giao diện



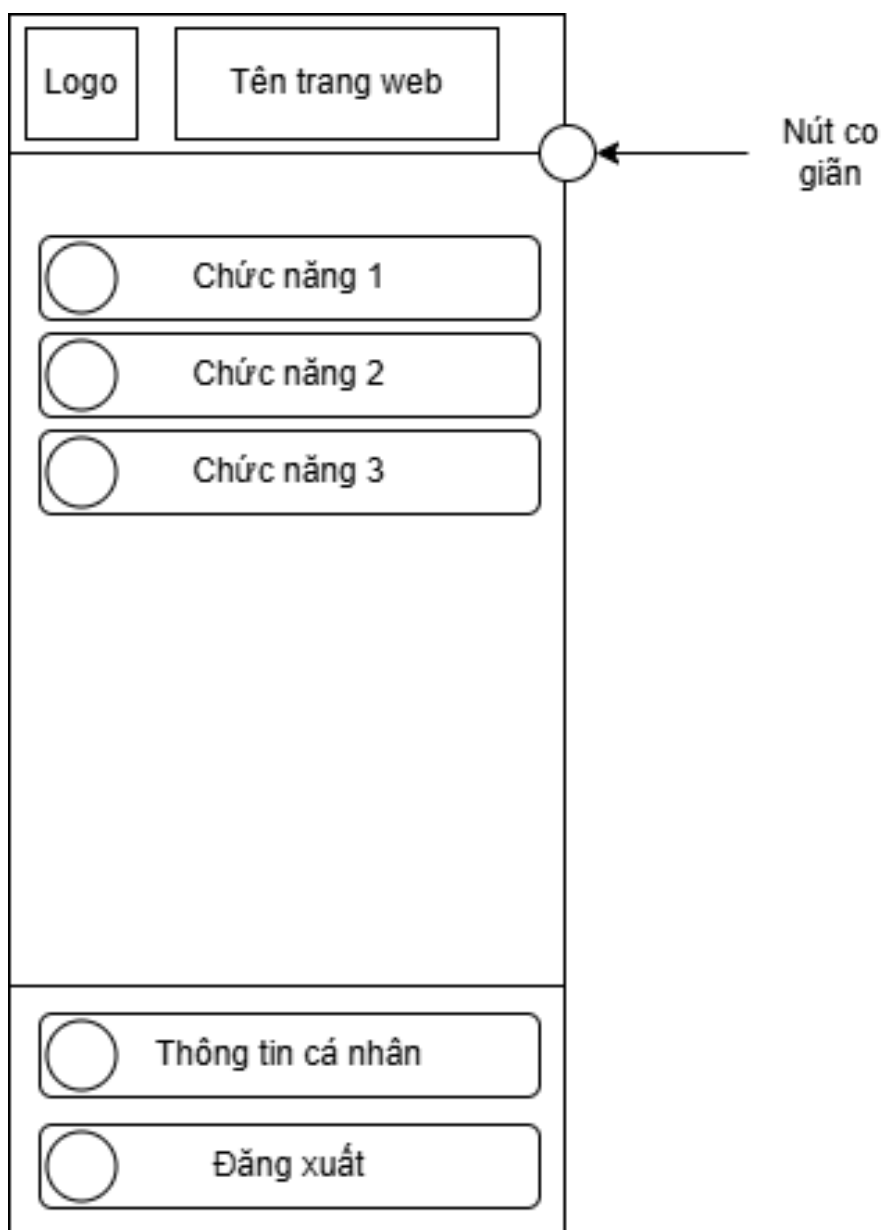
Hình 4.4: Bố cục màn hình của nhân viên và quản lý

Hình 4.4 mô tả bố cục màn hình của nhân viên và quản lý khách sạn. Giao diện gồm có 3 phần chính là header, sidebar và content. Phần header nằm ở trên cùng và chứa thông tin người dùng như ảnh đại diện, họ tên và vai trò trong công việc (quản trị viên hoặc nhân viên) như hình bên dưới.



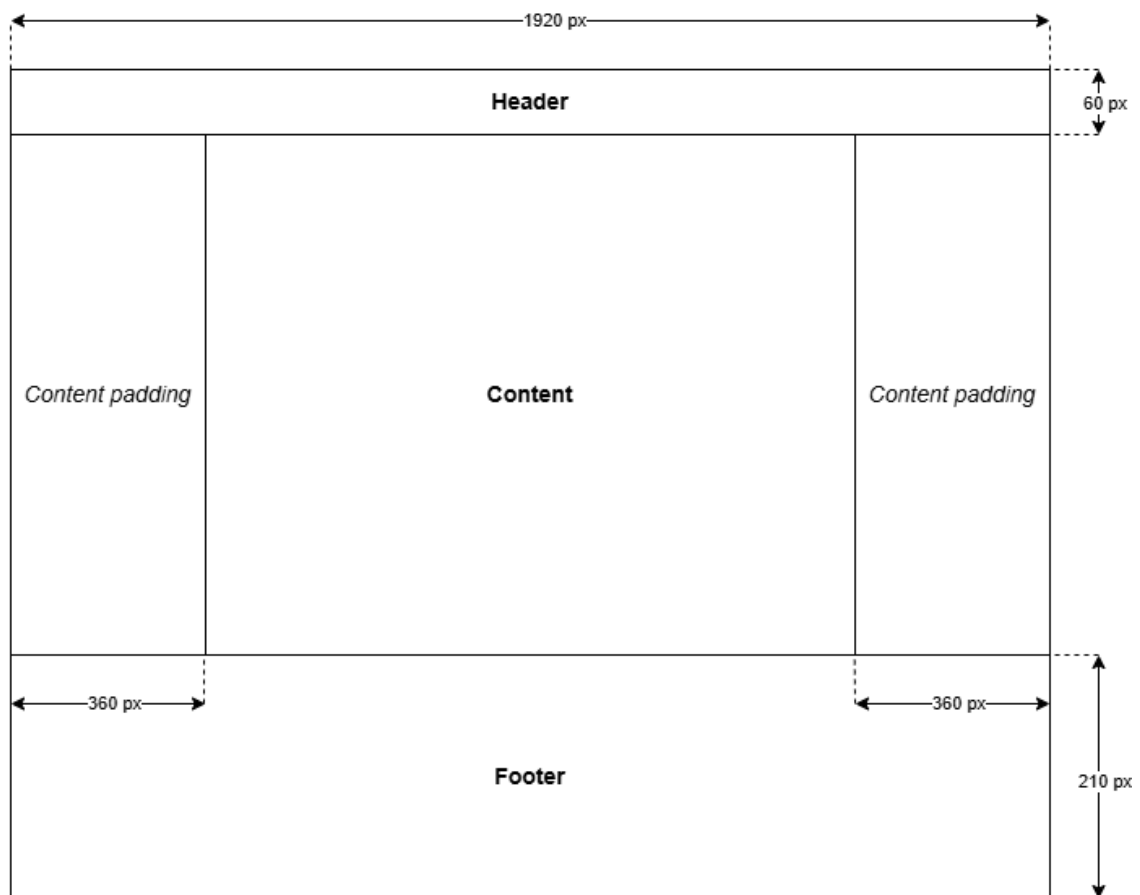
Hình 4.5: Bố cục header của màn hình nhân viên và quản lý

Phần sidebar nằm ở bên trái màn hình với chiều rộng là 250px và có thể co giãn. Khi co lại có chiều rộng là 60px. Ở phần trên có logo và tên trang web. Ngay dưới là các nút điều hướng tới các chức năng. Và ở dưới cùng có các nút thao tác như thông tin cá nhân hoặc đăng xuất.



Hình 4.6: Bố cục sidebar của màn hình của nhân viên và quản lý

Tiếp theo là giao diện cho khách hàng sử dụng dịch vụ.



Hình 4.7: Bố cục màn hình khách hàng

Đối với giao diện khách hàng thì sẽ không có sidebar và thay vào đó là footer chứa các nội dung nhanh về trang web. Header của trang gồm 3 phần chính là logo, các nút điều hướng và ảnh đại diện. Khi ấn vào ảnh đại diện sẽ hiển thị một số chức năng như xem đơn đặt, đăng xuất.

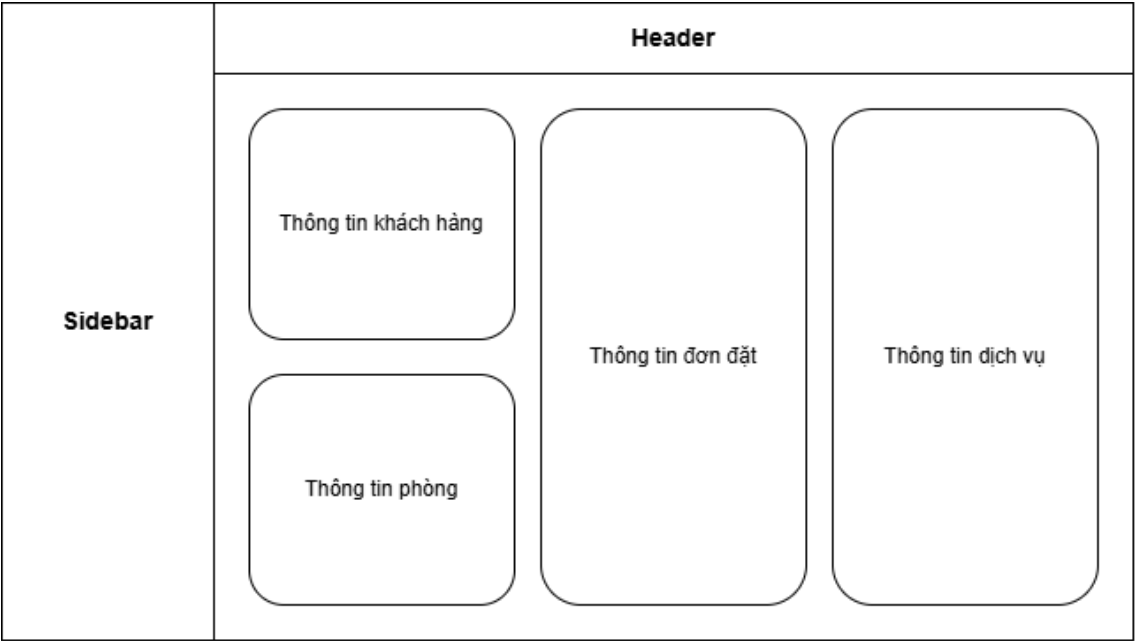


Hình 4.8: Bố cục header của màn hình khách hàng

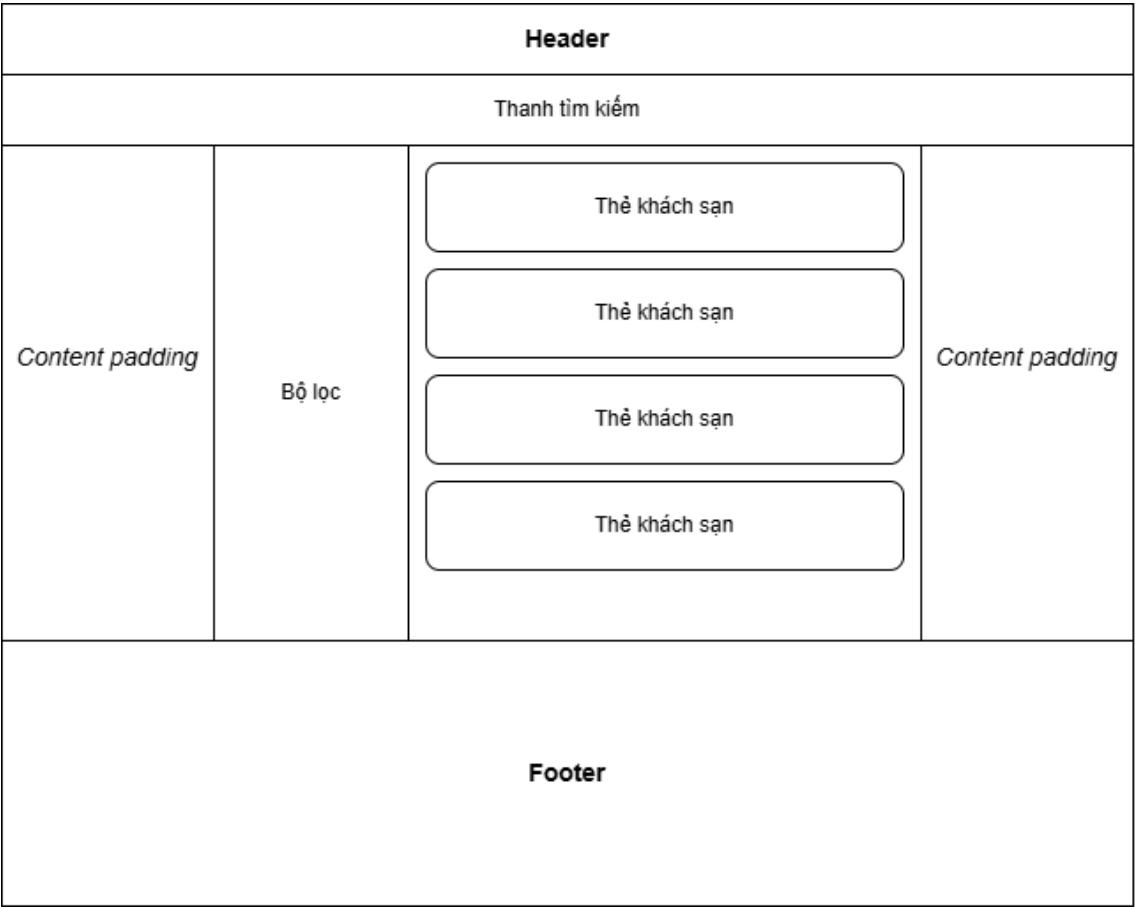
Phần content của trang dành cho người dùng thì được cách lề một khoảng 10% chiều rộng của trang.

c, Màn hình minh họa

Do các màn được thiết kế cùng bố cục chỉ khác một chút ở phần nội dung, vì vậy em sẽ trình bày 2 ví dụ minh họa cho 2 bố cục đã trình bày ở trên.



Hình 4.9: Màn hình minh họa xem chi tiết đơn đặt của khách hàng



Hình 4.10: Màn hình minh họa tìm kiếm khách sạn

4.2.2 Thiết kế lớp

Ở phần này em sẽ trình bày thiết kế lớp chi tiết cho 3 lớp quan trọng nhất là BookingController, RoomController và HotelController.

<i>BookingController</i>
createBooking
createBookingByAdmin
createServiceBooking
updateBooking
getBookingsByUserId
getBookingsByHotelId
getBookingById
getBookingByAdmin

Hình 4.11: Thiết kế chi tiết lớp BookingController

Lớp BookingController có nhiệm vụ xử lý các request liên quan tới đặt phòng và đặt dịch vụ. Cụ thể các phương thức như sau:

- **createBooking()**: Phương thức này dùng để tạo một đặt phòng mới bởi khách hàng.
- **createBookingByAdmin()**: Phương thức này dùng để quản trị viên tạo một đặt phòng mới.
- **createServiceBooking()**: Phương thức này dùng để tạo một đặt dịch vụ mới.
- **updateBooking()**: Phương thức này dùng để cập nhật thông tin của một đặt phòng hiện có.
- **getBookingsByUserId()**: Phương thức này dùng để lấy danh sách các đặt phòng dựa trên ID của người dùng.
- **getBookingsByHotelId()**: Phương thức này dùng để lấy danh sách các đặt phòng dựa trên ID của khách sạn.
- **getBookingById()**: Phương thức này dùng để lấy thông tin của một đặt phòng cụ thể dựa trên ID của đặt phòng đó.
- **getBookingByAdmin()**: Phương thức này dùng để quản trị viên lấy thông tin đặt phòng.

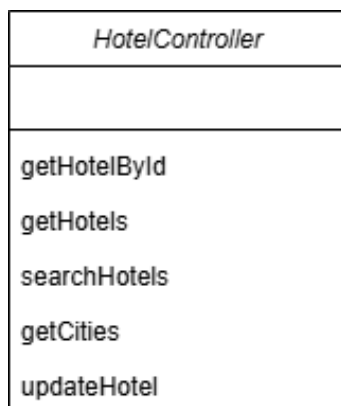
<i>RoomController</i>
getRoomTypes getRooms getRoomTypeById getRoomById createRoomType createRoom updateRoomType updateRoom deleteRoomType deleteRoom

Hình 4.12: Thiết kế chi tiết lớp RoomController

Lớp RoomController có nhiệm vụ xử lý các request liên quan tới phòng và loại phòng. Cụ thể các phương thức như sau:

- **getRoomTypes():** Phương thức này dùng để lấy tất cả các loại phòng hiện có của khách sạn.
- **getRooms():** Phương thức này dùng để lấy tất cả các phòng hiện có của loại phòng đã chọn.
- **getRoomTypeById():** Phương thức này dùng để lấy thông tin chi tiết của một loại phòng cụ thể.
- **getRoomById():** Phương thức này dùng để lấy thông tin chi tiết của một phòng cụ thể.
- **createRoomType():** Phương thức này dùng để tạo một loại phòng mới.
- **createRoom():** Phương thức này dùng để tạo một phòng mới.
- **updateRoomType():** Phương thức này dùng để cập nhật thông tin của một loại phòng đã tồn tại.
- **updateRoom():** Phương thức này dùng để cập nhật thông tin của một phòng đã tồn tại.
- **deleteRoomType():** Phương thức này dùng để xóa một loại phòng khỏi hệ thống.

- **deleteRoom()**: Phương thức này dùng để xóa một phòng khỏi hệ thống.

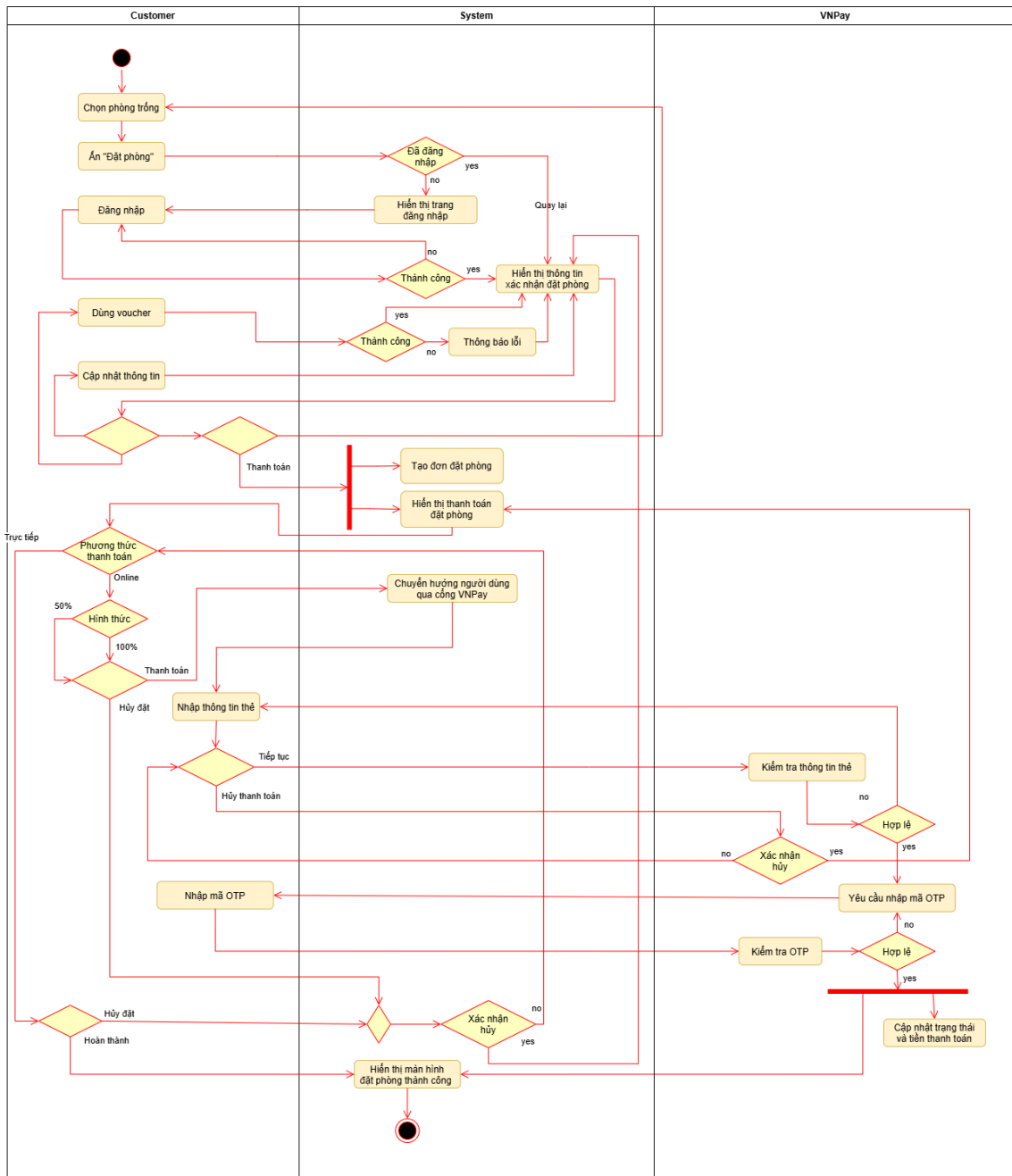


Hình 4.13: Thiết kế chi tiết lớp HotelController

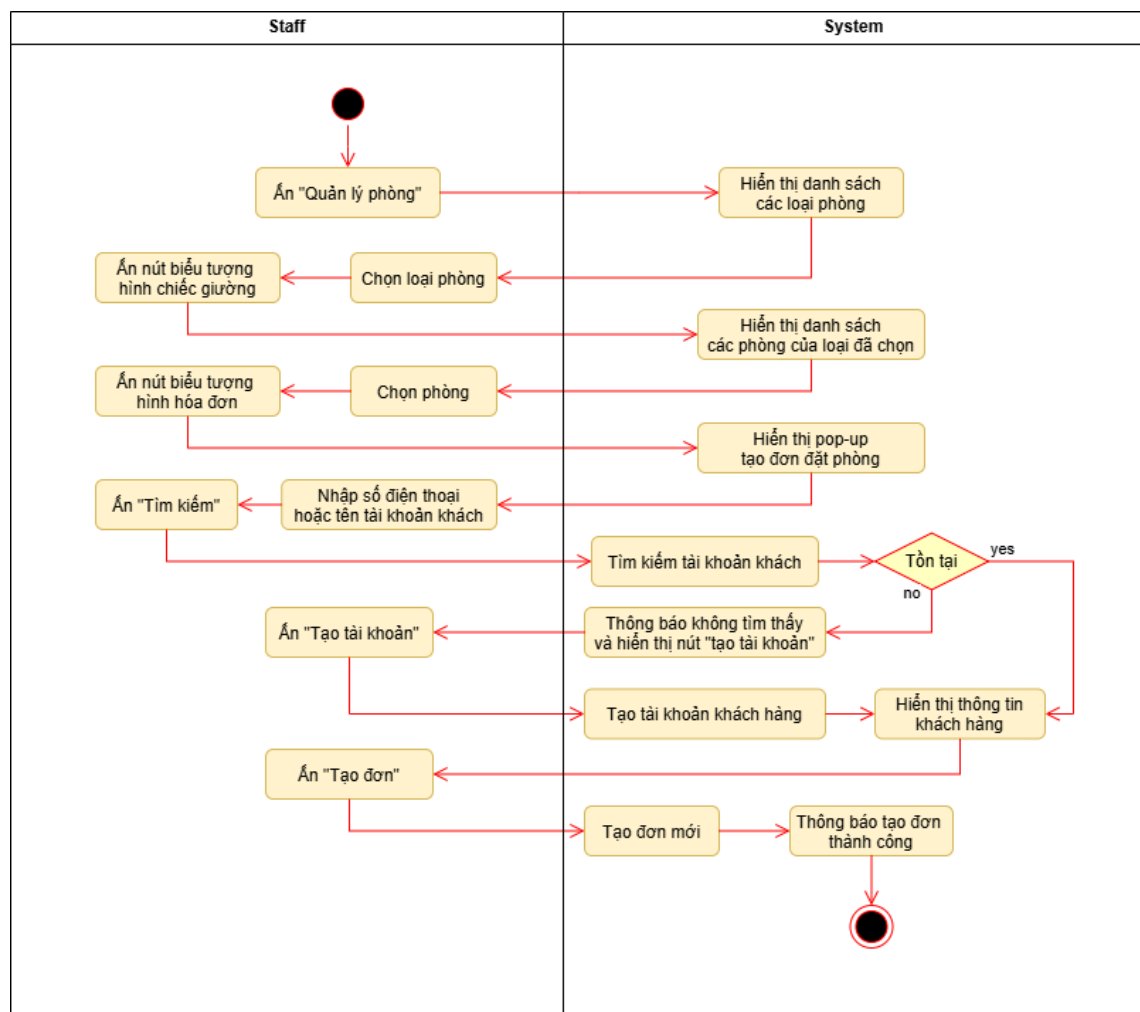
Lớp HotelController có nhiệm vụ xử lý các request liên quan tới khách sạn. Cụ thể các phương thức như sau:

- **getHotelById()**: Phương thức này dùng để lấy thông tin chi tiết của một khách sạn cụ thể, thường là thông qua một định danh (ID) duy nhất của khách sạn đó.
- **getHotels()**: Phương thức này dùng để lấy tất cả các khách sạn hiện có trong hệ thống.
- **searchHotels()**: Phương thức này dùng để tìm kiếm các khách sạn dựa trên các tiêu chí nhất định, ví dụ như tên, địa điểm, tiện nghi, v.v.
- **getCities()**: Phương thức này dùng để lấy danh sách các thành phố hiện có khách sạn trong hệ thống.
- **updateHotel()**: Phương thức này dùng để cập nhật thông tin của một khách sạn đã tồn tại trong hệ thống.

Để minh họa thiết kế lớp, em xin trình bày luồng truyền thông điệp bằng biểu đồ trình tự của 2 use case chính là đặt phòng (khách hàng) và thêm đơn đặt phòng (nhân viên) .



Hình 4.14: Biểu đồ trình tự use case đặt phòng



Hình 4.15: Biểu đồ trình tự use case thêm đơn đặt phòng

4.2.3 Thiết kế cơ sở dữ liệu

Phần này có độ dài từ hai đến bốn trang. Sinh viên thiết kế, vẽ và giải thích biểu đồ thực thể liên kết (E-R diagram). Từ đó, sinh viên thiết kế cơ sở dữ liệu tùy theo hệ quản trị cơ sở dữ liệu mà mình sử dụng (SQL, NoSQL, Firebase, v.v.)

a, Biểu đồ thực thể liên kết



Hình 4.16: Biểu đồ thực thể liên kết

Các thực thể chính là các bảng chính trong cơ sở dữ liệu, đại diện cho các đối tượng quan trọng trong hệ thống:

- **users**: Thông tin về người dùng/khách hàng.
- **staffs**: Thông tin về nhân viên.
- **hotels**: Thông tin chi tiết về các khách sạn.

- **rooms:** Thông tin về từng phòng cụ thể trong khách sạn.
- **room_types:** Các loại phòng khác nhau (ví dụ: phòng đơn, phòng đôi).
- **bookings:** Các giao dịch đặt phòng của khách hàng.
- **services:** Các dịch vụ bổ sung mà khách sạn cung cấp (ví dụ: spa, ăn sáng).
- **reviews:** Các đánh giá của người dùng về khách sạn/dịch vụ.
- **vouchers:** Mã giảm giá.
- **amenities:** Các tiện nghi có sẵn (ví dụ: Wi-Fi, hồ bơi).
- **revenues:** Dữ liệu doanh thu.
- **predicts:** Dữ liệu dự đoán (ví dụ: số lượng khách).

Các mối quan hệ chính thể hiện cách các thực thể tương tác và liên kết với nhau:

- **Người dùng và Đặt phòng/Đánh giá:** Một người dùng có thể tạo nhiều đặt phòng và viết nhiều đánh giá.
- **Khách sạn và Phòng/Dịch vụ/Đặt phòng/Đánh giá:** Một khách sạn có nhiều phòng, cung cấp nhiều dịch vụ, nhận nhiều đặt phòng và nhận nhiều đánh giá.
- **Đặt phòng và Phòng/Voucher/Dịch vụ:** Mỗi đặt phòng liên quan đến một phòng cụ thể, có thể sử dụng một voucher, và có thể bao gồm nhiều dịch vụ bổ sung.
- **Phòng và Loại phòng:** Mỗi phòng thuộc về một loại phòng nhất định.
- **Khách sạn/Loại phòng và Tiện nghi:** Khách sạn và loại phòng có thể có nhiều tiện nghi, và một tiện nghi có thể thuộc về nhiều khách sạn/loại phòng.

Mục đích tổng thể: Biểu đồ này phác thảo cấu trúc cơ sở dữ liệu cho một hệ thống quản lý và đặt phòng khách sạn. Nó cho phép hệ thống:

- Lưu trữ thông tin chi tiết về khách sạn, phòng, dịch vụ và tiện nghi.
- Quản lý thông tin người dùng và nhân viên.
- Xử lý các giao dịch đặt phòng, bao gồm cả việc áp dụng voucher và thêm dịch vụ.
- Thu thập và hiển thị đánh giá của khách hàng.
- Theo dõi doanh thu và dự đoán.

Đây là một cái nhìn tổng quan về cách các thành phần chính của hệ thống được kết nối với nhau trong cơ sở dữ liệu, đảm bảo tính toàn vẹn và hiệu quả của dữ liệu.

b, Thiết kế chi tiết

Dưới đây là thiết kế chi tiết cho các bảng trong database.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho người dùng. (Khóa chính - PK)
username	VARCHAR (255)	Tên đăng nhập của người dùng. (Không rỗng - NN)
password	VARCHAR (255)	Mật khẩu đã được mã hóa của người dùng. (Không rỗng - NN)
first_name	VARCHAR (255)	Tên của người dùng.
last_name	VARCHAR (255)	Họ của người dùng.
avatar	TEXT	URL hoặc đường dẫn đến ảnh đại diện của người dùng.
phone	VARCHAR (100)	Số điện thoại của người dùng.
email	VARCHAR (255)	Địa chỉ email của người dùng.
voucher_id	INT	Mã định danh của voucher mặc định hoặc cuối cùng được sử dụng. (Khóa ngoại - FK tới vouchers.id)
created_at	DATETIME	Thời gian tạo bản ghi người dùng.
updated_at	DATETIME	Thời gian cập nhật bản ghi người dùng lần cuối.

Bảng 4.2: Mô tả bảng `users`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho nhân viên. (Khóa chính - PK)
username	VARCHAR (255)	Tên đăng nhập của nhân viên. (Không rỗng - NN)
password	VARCHAR (255)	Mật khẩu đã được mã hóa của nhân viên. (Không rỗng - NN)
first_name	VARCHAR (255)	Tên của nhân viên.
last_name	VARCHAR (255)	Họ của nhân viên.
avatar	VARCHAR (255)	URL hoặc đường dẫn đến ảnh đại diện của nhân viên.
phone	VARCHAR (100)	Số điện thoại của nhân viên.
email	VARCHAR (255)	Địa chỉ email của nhân viên.
address	VARCHAR (255)	Địa chỉ của nhân viên.
role	ENUM	Vai trò của nhân viên (ví dụ: admin, user). (Không rỗng - NN)
created_at	DATETIME	Thời gian tạo bản ghi nhân viên.
updated_at	DATETIME	Thời gian cập nhật bản ghi nhân viên lần cuối.

Bảng 4.3: Mô tả bảng `staffs`.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho khách sạn. (Khóa chính - PK)
name	VARCHAR (255)	Tên của khách sạn. (Không rỗng - NN)
avatar	TEXT	URL hoặc đường dẫn đến ảnh đại diện của khách sạn.
images	TEXT	Chuỗi JSON hoặc danh sách các URL ảnh của khách sạn.
description	TEXT	Mô tả chi tiết về khách sạn.
address	TEXT	Địa chỉ vật lý của khách sạn.
city	VARCHAR (255)	Thành phố nơi khách sạn tọa lạc.
phone	VARCHAR (100)	Số điện thoại liên hệ của khách sạn.
email	VARCHAR (255)	Địa chỉ email liên hệ của khách sạn.
min_price	DECIMAL (10, 0)	Giá phòng thấp nhất có sẵn tại khách sạn.
rating	FLOAT	Đánh giá trung bình của khách sạn.
predict	DECIMAL (10, 0)	Số lượng khách dự kiến cho khách sạn.
lat	DECIMAL (10, 7)	Vĩ độ địa lý của khách sạn.
lon	DECIMAL (10, 7)	Kinh độ địa lý của khách sạn.

Bảng 4.4: Mô tả bảng `hotels`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho đánh giá. (Khóa chính - PK)
user_id	INT	Mã định danh của người dùng đã viết đánh giá. (Khóa ngoại - FK tới <code>users.id</code> , Không rỗng - NN)
hotel_id	INT	Mã định danh của khách sạn được đánh giá. (Khóa ngoại - FK tới <code>hotels.id</code> , Không rỗng - NN)
booking_id	INT	Mã định danh của đặt phòng liên quan đến đánh giá (nếu có). (Khóa ngoại - FK tới <code>bookings.id</code>)
rating	INT	Điểm đánh giá (ví dụ: từ 1 đến 5). (Không rỗng - NN)
comment	TEXT	Nội dung bình luận của đánh giá.
images	TEXT	Chuỗi JSON hoặc danh sách các URL ảnh đi kèm đánh giá.
created_at	DATETIME	Thời gian tạo bản ghi đánh giá.
updated_at	DATETIME	Thời gian cập nhật bản ghi đánh giá lần cuối.

Bảng 4.5: Mô tả bảng `reviews`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho đặt phòng. (Khóa chính - PK)
user_id	INT	Mã định danh của người dùng đã đặt phòng. (Khóa ngoại - FK tới <code>users.id</code> , Không rỗng - NN)
room_id	INT	Mã định danh của phòng được đặt. (Khóa ngoại - FK tới <code>rooms.id</code> , Không rỗng - NN)
hotel_id	INT	Mã định danh của khách sạn được đặt. (Khóa ngoại - FK tới <code>hotels.id</code> , Không rỗng - NN)
check_in	DATETIME	Thời gian nhận phòng dự kiến. (Không rỗng - NN)
check_out	DATETIME	Thời gian trả phòng dự kiến. (Không rỗng - NN)
room_price	DECIMAL(10, 0)	Giá phòng tại thời điểm đặt.
voucher_id	INT	Mã định danh của voucher đã được áp dụng cho đặt phòng. (Khóa ngoại - FK tới <code>vouchers.id</code>)
room_amount	DECIMAL(10, 0)	Tổng số tiền cho phòng.
service_amount	DECIMAL(10, 0)	Tổng số tiền cho các dịch vụ bổ sung.
total_amount	DECIMAL(10, 0)	Tổng số tiền trước khi áp dụng giảm giá.
paid_amount	DECIMAL(10, 0)	Số tiền đã thanh toán.
final_amount	DECIMAL(10, 0)	Số tiền cuối cùng cần thanh toán sau khi áp dụng tất cả giảm giá.
status	ENUM	Trạng thái của đặt phòng (ví dụ: <code>pending</code> , <code>confirmed</code> , <code>cancelled</code> , <code>expired</code>).
checked_in_at	DATETIME	Thời gian thực tế khách nhận phòng.
expired_at	DATETIME	Thời gian hết hạn của đặt phòng (nếu có).
created_at	DATETIME	Thời gian tạo bản ghi đặt phòng.
updated_at	DATETIME	Thời gian cập nhật bản ghi đặt phòng lần cuối.

Bảng 4.6: Mô tả bảng `bookings`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho voucher. (Khóa chính - PK)
code	VARCHAR (255)	Mã code của voucher. (Không rỗng - NN)
name	VARCHAR (255)	Tên mô tả của voucher.
type	ENUM	Loại voucher (ví dụ: percent, fixed_amount). (Không rỗng - NN)
description	TEXT	Mô tả chi tiết về voucher.
discount	DECIMAL (10, 0)	Giá trị giảm giá của voucher. (Không rỗng - NN)
discount_type	ENUM	Loại giảm giá (ví dụ: percent, fixed_amount).
start_date	DATE	Ngày bắt đầu hiệu lực của voucher.
end_date	DATE	Ngày kết thúc hiệu lực của voucher.
status	ENUM	Trạng thái của voucher (ví dụ: active, expired, upcoming).

Bảng 4.7: Mô tả bảng vouchers.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho phòng. (Khóa chính - PK)
hotel_id	INT	Mã định danh của khách sạn sở hữu phòng này. (Khóa ngoại - FK tới hotels.id, Không rỗng - NN)
room_type_id	INT	Mã định danh của loại phòng. (Khóa ngoại - FK tới room_types.id, Không rỗng - NN)
room_number	VARCHAR (10)	Số phòng hoặc tên phòng.
status	ENUM	Trạng thái của phòng (ví dụ: available, occupied, maintenance).
area	DECIMAL (10, 0)	Diện tích của phòng.

Bảng 4.8: Mô tả bảng rooms.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho loại phòng. (Khóa chính - PK)
name	VARCHAR (255)	Tên của loại phòng (ví dụ: "Standard", "Deluxe", "Suite"). (Không rỗng - NN)
images	TEXT	Chuỗi JSON hoặc danh sách các URL ảnh của loại phòng.
description	TEXT	Mô tả chi tiết về loại phòng.
price	DECIMAL (10, 0)	Giá cơ bản của loại phòng. (Không rỗng - NN)
capacity	INT	Sức chứa tối đa của loại phòng.
area	DECIMAL (4, 1)	Diện tích trung bình của loại phòng.
hotel_id	INT	Mã định danh của khách sạn mà loại phòng này thuộc về (nếu loại phòng là duy nhất cho khách sạn). (Khóa ngoại - FK tới hotels.id)

Bảng 4.9: Mô tả bảng room_types.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho tiện nghi. (Khóa chính - PK)
name	VARCHAR (255)	Tên của tiện nghi (ví dụ: "Wifi", "Hồ bơi", "Điều hòa"). (Không rỗng - NN)
category	ENUM	Danh mục của tiện nghi (ví dụ: bathroom, room, general).

Bảng 4.10: Mô tả bảng amenities.

Thuộc tính	Kiểu dữ liệu	Mô tả
hotel_id	INT	Mã định danh của khách sạn. (Khóa chính & Khóa ngoại - PK, FK tới hotels.id, Không rỗng - NN)
amenity_id	INT	Mã định danh của tiện nghi. (Khóa chính & Khóa ngoại - PK, FK tới amenities.id, Không rỗng - NN)

Bảng 4.11: Mô tả bảng hotel_amenities.

Thuộc tính	Kiểu dữ liệu	Mô tả
hotel_id	INT	Mã định danh của khách sạn. (Khóa chính & Khóa ngoại - PK, FK tới hotels.id, Không rỗng - NN)
amenity_id	INT	Mã định danh của tiện nghi. (Khóa chính & Khóa ngoại - PK, FK tới amenities.id, Không rỗng - NN)

Bảng 4.12: Mô tả bảng hotel_amenities.

Thuộc tính	Kiểu dữ liệu	Mô tả
room_type_id	INT	Mã định danh của loại phòng. (Khóa chính & Khóa ngoại - PK, FK tới room_types.id, Không rỗng - NN)
amenity_id	INT	Mã định danh của tiện nghi. (Khóa chính & Khóa ngoại - PK, FK tới amenities.id, Không rỗng - NN)

Bảng 4.13: Mô tả bảng room_type_amenities.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho dịch vụ. (Khóa chính - PK)
name	VARCHAR(255)	Tên của dịch vụ (ví dụ: "Bữa sáng", "Đưa đón sân bay"). (Không rỗng - NN)
images	TEXT	Chuỗi JSON hoặc danh sách các URL ảnh của dịch vụ.
description	TEXT	Mô tả chi tiết về dịch vụ.
category	ENUM	Danh mục của dịch vụ (ví dụ: dining, entertainment, facilities).
price	DECIMAL(10, 0)	Giá của dịch vụ.
hotel_id	INT	Mã định danh của khách sạn cung cấp dịch vụ này. (Khóa ngoại - FK tới hotels.id)
created_at	DATETIME	Thời gian tạo bản ghi dịch vụ.
updated_at	DATETIME	Thời gian cập nhật bản ghi dịch vụ lần cuối.

Bảng 4.14: Mô tả bảng services.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho việc đặt dịch vụ. (Khóa chính - PK)
booking_id	INT	Mã định danh của đặt phòng mà dịch vụ này thuộc về. (Khóa ngoại - FK tới <code>bookings.id</code> , Không rỗng - NN)
service_id	INT	Mã định danh của dịch vụ được đặt. (Khóa ngoại - FK tới <code>services.id</code> , Không rỗng - NN)
quantity	INT	Số lượng dịch vụ được đặt.
price	DECIMAL(10, 0)	Giá của dịch vụ tại thời điểm đặt.
voucher_id	INT	Mã định danh của voucher áp dụng riêng cho dịch vụ này (nếu có). (Khóa ngoại - FK tới <code>vouchers.id</code>)
total_amount	DECIMAL(10, 0)	Tổng số tiền cho dịch vụ này.
final_amount	DECIMAL(10, 0)	Số tiền cuối cùng cần thanh toán cho dịch vụ này.
status	ENUM	Trạng thái của đặt dịch vụ (ví dụ: <code>pending</code> , <code>confirmed</code> , <code>cancelled</code>).
created_at	DATETIME	Thời gian tạo bản ghi đặt dịch vụ.
updated_at	DATETIME	Thời gian cập nhật bản ghi đặt dịch vụ lần cuối.

Bảng 4.15: Mô tả bảng `service_bookings`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho bản ghi doanh thu. (Khóa chính - PK)
hotel_id	INT	Mã định danh của khách sạn. (Khóa ngoại - FK tới <code>hotels.id</code> , Không rỗng - NN)
report_date	DATE	Ngày báo cáo doanh thu. (Không rỗng - NN)
revenue	DECIMAL(10, 0)	Số tiền doanh thu. (Không rỗng - NN)
type	ENUM	Loại báo cáo (ví dụ: <code>daily</code> , <code>monthly</code> , <code>yearly</code>). (Không rỗng - NN)
created_at	DATETIME	Thời gian tạo bản ghi doanh thu.
updated_at	DATETIME	Thời gian cập nhật bản ghi doanh thu lần cuối.

Bảng 4.16: Mô tả bảng `revenues`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho bản ghi dự đoán. (Khóa chính - PK)
hotel_id	INT	Mã định danh của khách sạn liên quan. (Khóa ngoại - FK tới <code>hotels.id</code> , Không rỗng - NN)
date	DATE	Ngày dự đoán. (Không rỗng - NN)
predicted_guests	INT	Số lượng khách dự kiến.
actual_guests	INT	Số lượng khách thực tế (để so sánh).

Bảng 4.17: Mô tả bảng `predicts`.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	INT	Mã định danh duy nhất cho bản ghi user-voucher. (Khóa chính - PK)
user_id	INT	Mã định danh của người dùng sở hữu voucher. (Khóa ngoại - FK tới <code>users.id</code> , Không rỗng - NN)
voucher_id	INT	Mã định danh của voucher. (Khóa ngoại - FK tới <code>vouchers.id</code> , Không rỗng - NN)
created_at	DATETIME	Thời gian bản ghi được tạo.
updated_at	DATETIME	Thời gian bản ghi được cập nhật lần cuối.

Bảng 4.18: Mô tả bảng `user_vouchers`.

4.2.4 Thiết kế module học máy

Module học máy trong hệ thống có vai trò hỗ trợ dự đoán lượng khách đặt phòng mỗi ngày, từ đó giúp khách sạn chủ động điều phối nguồn lực, tối ưu hóa dịch vụ và hỗ trợ ra quyết định kinh doanh. Module này được tách biệt dưới dạng một dịch vụ riêng biệt (`machine_learning`) giao tiếp với backend chính thông qua API, tuân thủ nguyên tắc phân tầng rõ ràng và dễ bảo trì.

a, Thiết kế tổng quát

Module được thiết kế theo mô hình client-server, trong đó:

- **Client (Backend - hệ thống chính):** gửi yêu cầu dự đoán thông qua HTTP request (POST).
- **Server (Machine Learning):** được xây dựng bằng Python và Flask, nhận dữ liệu đầu vào, xử lý dữ liệu, gọi model học máy đã huấn luyện, trả kết quả dự đoán.

b, Module API

Module này chịu trách nhiệm khởi tạo ứng dụng Flask và định nghĩa các endpoint dùng để giao tiếp với hệ thống backend (Express). Khi backend gửi dữ liệu

đầu vào, module này sẽ nhận, chuyển cho module dự đoán, sau đó trả về kết quả. Có 2 api chính như sau:

```
@predict_bp.route("/predict/<int:hotel_id>", methods=["POST"])
def predict(hotel_id):
    try:
        input_data = request.get_json()["input"]
        result = predict_guests(hotel_id, input_data)
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Hình 4.17: Api dự đoán lượng khách

```
@train_bp.route("/train", methods=["POST"])
def train_model():
    try:
        # Tạo folder nếu chưa tồn tại
        save_folder = os.path.join(os.getcwd(), "data", "raw")
        os.makedirs(save_folder, exist_ok=True)

        # Đường dẫn lưu file
        save_path = os.path.join(save_folder, file.filename)

        # Lưu file
        file.save(save_path)

        # Đọc CSV từ file vừa lưu
        df = pd.read_csv(save_path)
        num_rows = len(df)
        print(f"File saved to {save_path}, rows: {num_rows}")

        preprocess(hotel_id, df)
        rf_result = train_random_forest(hotel_id)
        xgb_result = train_xgboost(hotel_id)
        sarima_result = train_sarima(hotel_id)

        return jsonify(
            {
                "success": True,
                "message": "Training thành công models",
                "random_forest": rf_result,
                "xgboost": xgb_result,
                "sarima": sarima_result,
            }
        )
```

Hình 4.18: Api huấn luyện mô hình

Vai trò: Là cầu nối giữa hệ thống chính và mô hình học máy.

Luồng xử lý:

1. Nhận dữ liệu gửi từ backend bằng phương thức POST.
2. Gọi hàm xử lý và thực hiện dự đoán.
3. Trả kết quả dưới dạng JSON cho backend.

c, Module Tiền xử lý dữ liệu

Chuyển đổi và chuẩn hóa dữ liệu đầu vào, tách cột `date` sang các định dạng `day`, `month`, `year`. Dưới đây là tập dữ liệu sau khi được xử lý.

Day ▼	Month ▲ ▼	Year ▼	Day_of_week ▼	Is_holiday ▼	Weather_condition ▼	Temperature ▼	Voucher_active ▼	Number_of_guests ▼
1	1	2025	Wednesday	1	Rain	24.8	0	25
2	1	2025	Thursday	0	Thunderstorm	36.6	0	15
3	1	2025	Friday	0	Rain	26.4	1	17
4	1	2025	Saturday	0	Clear	19.8	1	21
5	1	2025	Sunday	0	Clouds	28.8	1	36
6	1	2025	Monday	0	Thunderstorm	20.6	1	20
7	1	2025	Tuesday	0	Clouds	38.8	0	20
8	1	2025	Wednesday	0	Clouds	21.7	1	22
9	1	2025	Thursday	0	Thunderstorm	28.5	0	5
10	1	2025	Friday	0	Drizzle	24.2	0	16
11	1	2025	Saturday	0	Thunderstorm	30.4	1	35
12	1	2025	Sunday	0	Drizzle	38.8	0	23
13	1	2025	Monday	0	Rain	28.5	0	8
14	1	2025	Tuesday	0	Thunderstorm	42	0	6
15	1	2025	Wednesday	0	Clear	41.2	0	21
16	1	2025	Thursday	0	Clouds	24.4	1	32

Hình 4.19: Mẫu dữ liệu sau khi xử lý

Vai trò: Giúp đảm bảo dữ liệu đưa vào mô hình đúng định dạng và tương thích với mô hình đã huấn luyện.

Luồng xử lý:

1. Kiểm tra và xử lý định dạng ngày.
2. Xóa cột không cần đến như `lunar_date`.
3. Xóa những dòng dữ liệu rỗng nếu có.

d, Module Huấn luyện mô hình

Module này chịu trách nhiệm huấn luyện các mô hình học máy (Random Forest, XGBoost, SARIMA) với phương pháp chia 80% dữ liệu train và 20% dữ liệu test, đánh giá hiệu năng và lưu mô hình đã huấn luyện vào file để tái sử dụng sau này.

```
# Khởi tạo model
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Huấn luyện
model.fit(X_train, y_train)
```

Hình 4.20: Mô hình Random Forest

```
# Khởi tạo model XGBoost
model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,
    random_state=42,
    objective="reg:squarederror",
    n_jobs=-1,
)

# Huấn luyện
model.fit(X_train, y_train)
```

Hình 4.21: Mô hình XGBoost

```
# Khởi tạo model SARIMA
# (p,d,q) = (1,1,1), seasonal (P,D,Q,s) = (1,1,1,7) cho tuần
model = SARIMAX(y_train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 7))
model_fit = model.fit(dispatch=False)
```

Hình 4.22: Mô hình Sarima

Vai trò: Đây là bước quan trọng để tạo ra các mô hình dự đoán. Mô hình sau khi huấn luyện sẽ được dùng trong module Flask để phục vụ dự đoán lượng khách.

Luồng xử lý:

1. Lấy dữ liệu đã qua xử lý.

2. Chia 80% train và 20% test.
3. Tiến hành huấn luyện.
4. Lưu mô hình đã huấn luyện.
5. Đánh giá mô hình.

e, Module Dự đoán

Tải mô hình học máy từ file và thực hiện dự đoán dựa trên dữ liệu đầu vào đã tiền xử lý. Dữ liệu đầu vào sẽ được đưa về đúng định dạng và thứ tự các cột như khi huấn luyện và sẽ được mã hóa trước khi đưa vào dự đoán như hình bên dưới.

```

# Load model columns để đồng nhất thứ tự input
model_folder = os.path.join(os.getcwd(), "models")
model_columns_path = os.path.join(model_folder, "model_columns.pkl")
model_columns = joblib.load(model_columns_path)

# Chuyển input thành DataFrame
input_df = pd.DataFrame([input_data])

# One-hot encoding tương tự lúc training
input_encoded = pd.get_dummies(input_df)

# Thêm cột thiếu (nếu có) để khớp thứ tự cột
for col in model_columns:
    if col not in input_encoded.columns:
        input_encoded[col] = 0

# Đảm bảo đúng thứ tự cột
input_encoded = input_encoded[model_columns]

```

Hình 4.23: Quy trình xử lý dữ liệu trước khi dự đoán

Vai trò: Là thành phần cốt lõi để thực hiện dự đoán từ mô hình đã huấn luyện.

Luồng xử lý:

1. Nhận dữ liệu.
2. Xử lý dữ liệu.
3. Gọi các mô hình đã huấn luyện.
4. Tiến hành dự đoán.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Dưới đây là danh sách các ngôn ngữ lập trình, framework, API, thư viện và công cụ hỗ trợ được sử dụng trong quá trình phát triển hệ thống. Mỗi thành phần đều được trình bày rõ ràng về mục đích sử dụng, phiên bản cụ thể và đường dẫn tham khảo, đảm bảo tính minh bạch, khả năng tái sử dụng và dễ dàng tra cứu trong quá trình phát triển, kiểm thử và triển khai hệ thống:

Mục đích sử dụng	Công cụ	Phiên bản	URL chính thức
Lập trình frontend	JavaScript (ES6)	ES6 (2015)	developer.mozilla.org
Xây dựng giao diện người dùng	ReactJS	18.2.0	react.dev
Phát triển frontend hiệu quả	Vite	5.0.0	vitejs.dev
Lập trình backend	NodeJS	22.12.0	nodejs.com
Lập trình backend, tạo API	ExpressJS	4.18.2	expressjs.com
Kết nối backend với MySQL bằng ORM	Sequelize	6.32.1	sequelize.org
Cơ sở dữ liệu chính của hệ thống	MySQL	8.0.36	mysql.com
Truy vấn và thao tác dữ liệu	SQL	SQL:2016	wikipedia.org
Tạo API phục vụ mô hình học máy	Flask	2.3.3	flask.palletsprojects.com
Lập lịch và xử lý tác vụ định kỳ	node-cron	3.0.3	node-cron.com
Huấn luyện mô hình dự đoán (tree boosting)	XGBoost	2.0.3	xgboost.ai
Huấn luyện mô hình Random Forest	scikit-learn	1.4.2	scikit-learn.org
Phân tích chuỗi thời gian (SARIMA)	statsmodels	0.14.1	statsmodels.org
Xử lý và phân tích dữ liệu	pandas	2.2.2	pandas.pydata.org
Vẽ biểu đồ, trực quan hóa dữ liệu	matplotlib	3.8.4	matplotlib.org
Lưu và nạp mô hình học máy	joblib	1.4.0	joblib.readthedocs.io
Quản lý biến môi trường	dotenv	1.0.1	npmjs.com/package/dotenv
Gửi request từ frontend đến backend	axios	1.6.7	axios-http.com
Soạn thảo mã nguồn frontend/backend	Visual Studio Code	1.89.1	code.visualstudio.com
Viết và kiểm thử mã Python	PyCharm Community	2023.3.4	jetbrains.com/pycharm
Quản lý cơ sở dữ liệu MySQL	MySQL Workbench	8.0.36	dev.mysql.com/workbench
Kiểm thử API backend	Postman	10.24.2	postman.com
Phân tích và debug giao diện người dùng	Chrome DevTools	125.0	chrome.com/devtools
Dự báo thời tiết	OpenWeather API	v1	openweathermap.org/api
Chuyển đổi toạ độ sang địa điểm cụ thể	OpenCage Geocoding API	v1	opencagedata.com/api

Bảng 4.19: Thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Sau quá trình phân tích, thiết kế và xây dựng, đồ án đã hoàn thành một hệ thống hỗ trợ đặt phòng khách sạn thông minh, có khả năng dự đoán lượng khách mỗi

ngày bằng mô hình học máy, từ đó hỗ trợ khách sạn trong việc tối ưu hóa nguồn lực.

Sản phẩm đóng gói của đề án:

- **Frontend:** Giao diện người dùng được xây dựng bằng React, đáp ứng tốt trên trình duyệt hiện đại, cho phép khách hàng tra cứu, đặt phòng, sử dụng voucher, xem lịch sử đặt phòng, v.v.
- **Backend:** API được xây dựng bằng Express.js kết nối với cơ sở dữ liệu MySQL để xử lý các logic nghiệp vụ như xác thực, đặt phòng, quản lý người dùng, khách sạn, phòng, voucher...
- **Machine_learning:** Một dịch vụ riêng biệt được xây dựng bằng Flask để phục vụ việc dự đoán lượng khách hàng ngày dựa trên lịch sử đặt phòng và các yếu tố thời gian, loại phòng, khu vực.
- **Cơ sở dữ liệu MySQL:** Lưu trữ toàn bộ thông tin hệ thống một cách có tổ chức, đáp ứng tốt các truy vấn liên tục.

Ý nghĩa và vai trò của hệ thống

- Tối ưu hóa hoạt động vận hành khách sạn nhờ khả năng dự đoán trước nhu cầu lưu trú.
- Cung cấp nền tảng kỹ thuật hoàn chỉnh để mở rộng thành hệ thống đặt phòng thực tế.
- Đáp ứng đầy đủ nghiệp vụ đặt phòng từ frontend đến backend, hỗ trợ kiểm thử và triển khai thực tế.

Dưới đây là thông kê các thông tin về ứng dụng:

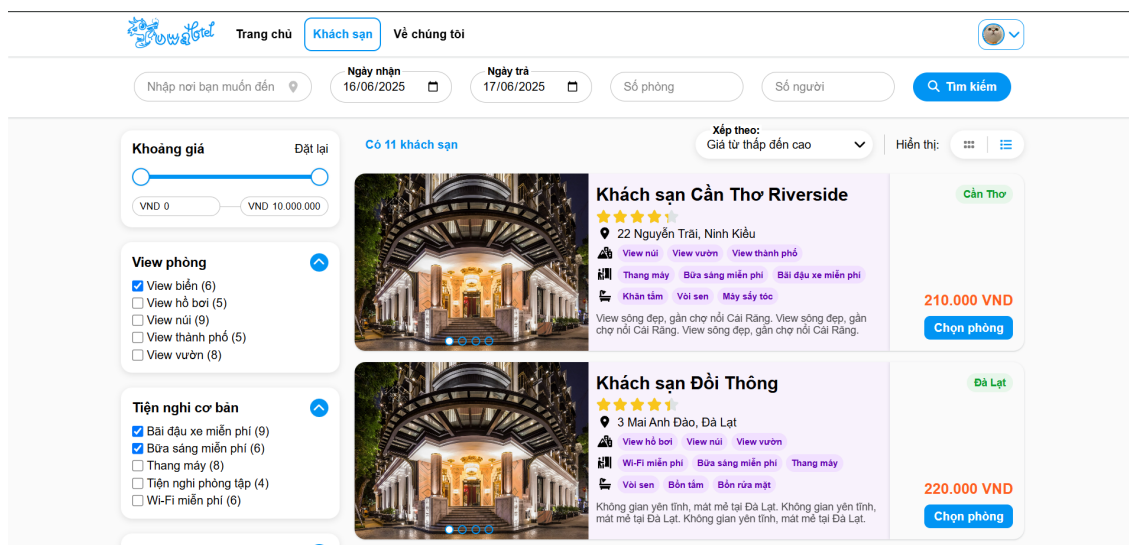
Thông tin thông kê	Giá trị	Ghi chú
Tổng số dòng code	42,280 dòng	Tính toàn bộ frontend, backend và machine learning
Số gói (package/module)	36 gói	Bao gồm tất cả các gói chức năng
Số lớp (class)	64 lớp	Tính cả class trong Express, React và Python
Dung lượng mã nguồn frontend	155 MB	Bao gồm source và thư viện React
Dung lượng mã nguồn backend	79.1 MB	Bao gồm Express app và các cấu hình liên quan
Dung lượng module machine learning	35.6 MB	Bao gồm Flask API và mô hình đã huấn luyện
Tổng dung lượng mã nguồn toàn hệ thống	269.7 MB	Tổng hợp tất cả phần frontend, backend và machine learning

Bảng 4.20: Thông kê thông tin ứng dụng

4.3.3 Minh họa các chức năng chính

Trong quá trình xây dựng hệ thống, em đã đặc biệt chú trọng đến thiết kế giao diện người dùng nhằm đảm bảo tính trực quan, dễ sử dụng và đáp ứng tốt nhu cầu thao tác thực tế. Trong phần này, em lựa chọn và trình bày các màn hình giao diện cho những chức năng chính, quan trọng và mang tính đặc trưng nhất của hệ thống.

Cụ thể, các giao diện bao gồm: giao diện tìm kiếm và đặt phòng khách sạn, giao diện duyệt đơn đặt phòng, giao diện dự đoán lượng khách lưu trú hằng ngày bằng mô hình học máy, giao diện quản lý khách sạn.



Hình 4.24: Giao diện tìm kiếm khách sạn

Đây là giao diện chính cho phép người dùng tìm kiếm và lọc các khách sạn/phòng dựa trên nhiều tiêu chí khác nhau.

Thanh tìm kiếm và Ngày nhận/trả phòng: Người dùng có thể nhập địa điểm mong muốn, chọn ngày nhận phòng và ngày trả phòng, số phòng và số người để tìm kiếm khách sạn phù hợp.

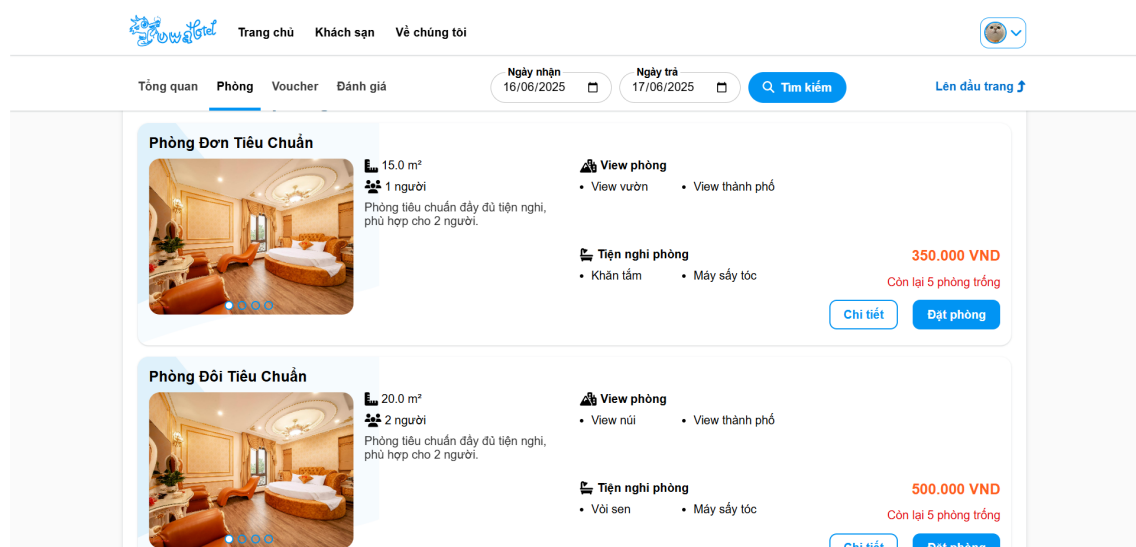
Bộ lọc bên trái:

- **Khoảng giá:** Cho phép người dùng kéo thanh trượt để chọn khoảng giá mong muốn, từ 0 VND đến 10.000.000 VND, hoặc đặt lại bộ lọc giá.
- **View phòng:** Người dùng có thể chọn loại view phòng ưa thích như View biển, View hồ bơi, View núi, View thành phố, hoặc View vườn.
- **Tiện nghi cơ bản:** Cung cấp các tùy chọn lọc theo tiện nghi như Bãi đậu xe miễn phí, Bữa sáng miễn phí, Tiện nghi phòng tập, và Wi-Fi miễn phí.

Khu vực hiển thị kết quả: Hiển thị danh sách các khách sạn phù hợp với tiêu chí tìm kiếm và lọc, ví dụ như “Khách sạn Cần Thơ Riverside” và “Khách sạn Đồi

Thông”. Mỗi kết quả hiển thị tên khách sạn, địa chỉ, đánh giá sao, các tiện ích nổi bật (ví dụ: Wi-Fi miễn phí, Bữa sáng miễn phí), giá và nút “Chọn phòng”.

Sắp xếp và Hiển thị: Người dùng có thể sắp xếp kết quả theo “Giá từ thấp đến cao” và chọn cách hiển thị danh sách (dạng lưới hoặc dạng danh sách chi tiết).



Hình 4.25: Giao diện chi tiết khách sạn

Giao diện này hiển thị thông tin chi tiết về các loại phòng có sẵn tại khách sạn, cho phép người dùng lựa chọn và đặt phòng.

Tìm kiếm & Điều hướng: Giao diện cung cấp các tab để điều hướng đến các mục như *Tổng quan*, *Phòng*, *Voucher*, và *Đánh giá*. Ngoài ra còn có thanh tìm kiếm tích hợp để chọn ngày nhận phòng, ngày trả phòng, số phòng và số người.

Danh sách loại phòng: Hiển thị danh sách các loại phòng hiện có (ví dụ: *Phòng Đơn Tiêu Chuẩn*, *Phòng Đôi Tiêu Chuẩn*) cùng với hình ảnh minh họa, mô tả ngắn gọn, giá phòng và số lượng phòng còn trống.

Hành động: Mỗi loại phòng đi kèm với hai nút chức năng: *Chi tiết* để xem thêm thông tin kỹ hơn và *Đặt phòng* để tiến hành quy trình đặt phòng.

The screenshot displays the 'Đặt phòng của bạn' (Your booking) section of a website. At the top, there's a navigation bar with 'Trang chủ', 'Khách sạn', and 'Về chúng tôi'. Below the navigation bar, the main heading is 'Đặt phòng của bạn' with a subtext: 'Hãy đảm bảo tất cả thông tin chi tiết trên trang này đã chính xác trước khi tiến hành thanh toán.' (Please ensure all details on this page are correct before proceeding with payment). The interface is divided into three main columns. The left column contains a form for 'Thông tin liên hệ' (Contact information) with fields for 'Họ*' (Last name), 'Tên*' (First name), 'Số điện thoại*' (Phone number), and 'Email*', each with a red asterisk indicating it's required. Below these fields is a 'Cập nhật' (Update) button. The middle column has a 'Xác thực thông tin' (Verify information) section with the text 'Sau khi kiểm tra các chính xác các thông tin cơ bản hãy đến bước tiếp theo' (After checking the accuracy of the basic information, please proceed to the next step). It includes a 'Quay lại' (Go back) button and a 'Thanh toán' (Pay) button. The right column displays the booking details for 'Khách sạn Cần Thơ Riverside' (Cần Thơ Riverside Hotel), which has a 4-star rating. It shows a photo of a hotel room, the room type 'Phòng Đơn Tiêu Chuẩn' (Standard Single Room), area '15.0 m²', occupancy '1 người' (1 person), dates 'Từ 16/06/2025 đến 17/06/2025 (1 đêm)' (From 16/06/2025 to 17/06/2025 (1 night)), and price '350.000 VND / đêm' (350,000 VND / night). At the bottom of this column are a 'Voucher' input field and an 'Áp dụng' (Apply) button.

Hình 4.26: Giao diện xác nhận đặt phòng

Giao diện này là bước đầu trong quy trình đặt phòng, cho phép người dùng xem lại và xác nhận thông tin trước khi tiến hành thanh toán.

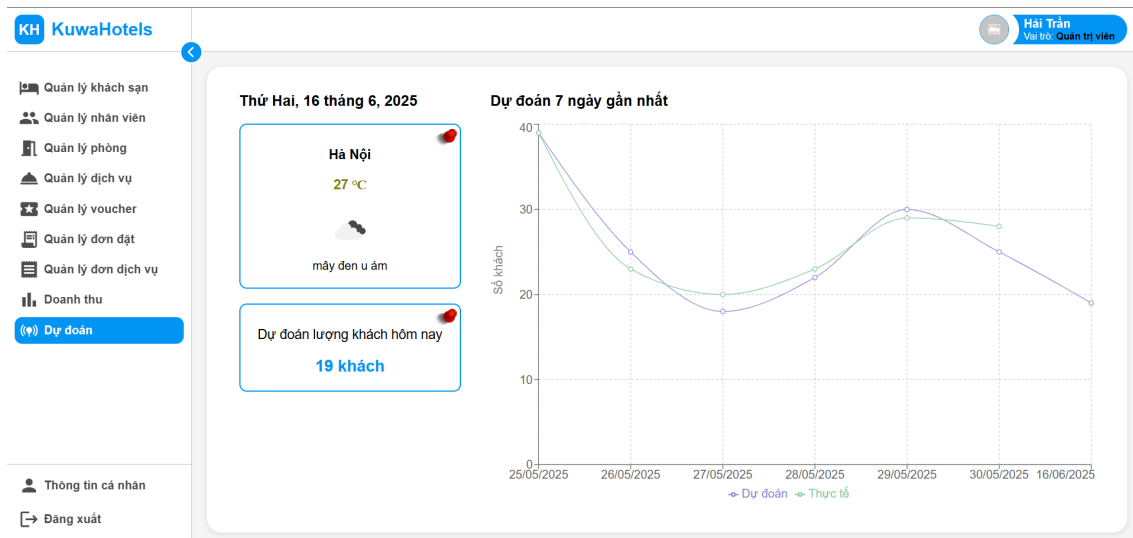
Trạng thái quy trình: Hiện thị tiến trình đặt phòng theo các bước: "*Đặt phòng và dịch vụ*" (đã hoàn thành), "*Thanh toán*", và "*Hoàn thành*".

Thông tin liên hệ: Người dùng có thể xem và cập nhật thông tin cá nhân như: Họ, Tên, Số điện thoại và Email. Sau khi cập nhật, có thể nhấn nút "*Cập nhật*".

Thông tin khách sạn và phòng: Hiện thị tên khách sạn (ví dụ: *Khách sạn Cần Thơ Riverside*), đánh giá sao, hình ảnh minh họa phòng, tên loại phòng (*Phòng Đơn Tiêu Chuẩn*), diện tích (15.0 m²), số người (1 người), ngày nhận/trả phòng (từ 16/06/2025 đến 17/06/2025 - 1 đêm), và tổng giá tiền (350.000 VND/đêm).

Xác thực thông tin và Thanh toán: Người dùng được yêu cầu kiểm tra lại các thông tin cơ bản. Có hai nút chức năng: "*Quay lại*" để chọn phòng khác và "*Thanh toán*" để hoàn tất quy trình.

Mục Voucher: Giao diện cung cấp một trường nhập mã giảm giá (*Voucher*) và nút "*Áp dụng*" để người dùng sử dụng khuyến mãi nếu có.



Hình 4.27: Giao diện dự đoán lượng khách hàng ngày

Đây là bảng điều khiển dành cho quản trị viên, cung cấp thông tin tổng quan về thời tiết và dự đoán số lượng khách. Giao diện bao gồm:

Thông tin thời tiết hiện tại: Hiển thị nhiệt độ (27°C) và tình trạng thời tiết (mây đen u ám) tại Hà Nội.

Dự đoán lượng khách hôm nay: Cung cấp con số dự đoán cụ thể cho ngày hiện tại, ví dụ: *19 khách*.

Biểu đồ dự đoán 7 ngày gần nhất: Biểu đồ đường thể hiện dữ liệu "Dự đoán" và "Thực tế" về số lượng khách trong 7 ngày gần đây, giúp theo dõi xu hướng và độ chính xác của mô hình.

Menu điều hướng: Thanh menu bên trái cho phép truy cập nhanh các chức năng quản lý như: Quản lý khách sạn, Quản lý nhân viên, Quản lý phòng, v.v.

Quản lý đơn đặt phòng

Tên khách hàng: Trạng thái: Ngày nhận:

STT	Tên khách hàng	Loại phòng	Số phòng	Trạng thái	Ngày nhận	Ngày trả	Xem	Hành động
1	Jane Smith	Phòng Đơn Tiêu Chuẩn	R6233	Đang chờ	19/06/2025	23/06/2025	👁	👍
2	Tung Tung Sahrur	Phòng Đơn Tiêu Chuẩn	R1514	Đang chờ	09/07/2025	13/07/2025	👁	👍
3	Tung Tung Sahrur	Phòng Deluxe	R6684	Đang chờ	20/06/2025	25/06/2025	👁	👍

Hình 4.28: Giao diện duyệt đơn đặt phòng

Đây là giao diện quản lý cho phép nhân viên xem và thao tác với các đơn đặt phòng của khách hàng.

Thanh tìm kiếm và bộ lọc: Gồm trường tìm kiếm theo “*Tên khách hàng*” và các bộ lọc theo “*Trạng thái*” (ví dụ: “*Đang chờ*”) và “*Ngày nhận*” (ví dụ: “*Tất cả*”) giúp tra cứu đơn đặt phòng nhanh chóng.

Danh sách đơn đặt phòng: Bảng dữ liệu hiển thị các thông tin chi tiết như: STT, Tên khách hàng, Loại phòng, Số phòng, Trạng thái, Ngày nhận, Ngày trả.

Hành động: Mỗi đơn đặt phòng có các nút:

- “*Xem*” – hiển thị chi tiết đơn đặt phòng.
- Biểu tượng *dấu tích* – xác nhận hoặc thực hiện thao tác duyệt đơn.

Menu điều hướng: Thanh menu bên trái cung cấp các tùy chọn quản lý khác như: Quản lý đơn đặt dịch vụ, Quản lý phòng, Quản lý dịch vụ, và Quản lý voucher.

The screenshot shows the 'Đơn đặt phòng #2' (Room Booking #2) interface. It features a sidebar with navigation options like 'Quản lý đơn đặt phòng', 'Quản lý đơn đặt dịch vụ', 'Quản lý phòng', 'Quản lý dịch vụ', and 'Quản lý voucher'. The main content area is divided into three sections: 'Thông tin khách hàng' (Customer Information), 'Thông tin đơn đặt' (Booking Information), and 'Thông tin dịch vụ (1)' (Service Information). The 'Thông tin khách hàng' section displays details for Jane Smith, including her phone number and email. The 'Thông tin đơn đặt' section shows the booking status as 'Đang chờ' (Pending), the room type as 'Phòng Đơn Tiêu Chuẩn' (Standard Single Room), and the booking dates. The 'Thông tin dịch vụ (1)' section contains a table with service details.

STT	Tên dịch vụ	Ngày dùng	SL
1	Ăn tối tự chọn	20/06/2025	1

Hình 4.29: Giao diện xem chi tiết đơn đặt phòng của khách hàng

Giao diện này cung cấp cái nhìn tổng quan và chi tiết về một đơn đặt phòng cụ thể của khách hàng như thông tin khách hàng, thông tin phòng, thông tin đơn đặt phòng và các dịch vụ đi kèm, giúp nhân viên dễ dàng quản lý.

4.4 Kiểm thử

Trong quá trình phát triển hệ thống, em đã tiến hành kiểm thử các chức năng chính nhằm đảm bảo chất lượng, độ tin cậy và tính ổn định của ứng dụng trước khi triển khai thực tế. Quá trình kiểm thử được thực hiện song song với quá trình xây dựng nhằm phát hiện sớm lỗi và điều chỉnh kịp thời, đặc biệt là ở các chức năng quan trọng như đặt phòng, xử lý thanh toán, dự đoán lượng khách và quản lý khuyến mãi.

Kỹ thuật kiểm thử chủ yếu được áp dụng là kiểm thử hộp đen (Black-box testing) – một kỹ thuật phổ biến trong kiểm thử phần mềm, tập trung vào kiểm tra các đầu vào và đầu ra của hệ thống mà không cần quan tâm đến cấu trúc hay mã nguồn bên trong. Với kỹ thuật này, em đặt mình vào vai trò của người dùng cuối để kiểm tra xem hệ thống có phản hồi đúng với các hành động mà người dùng thực hiện hay không.

STT	Trường hợp kiểm thử	Đầu vào	Kết quả mong đợi	Đánh giá
TC01	Thêm đơn đặt phòng thành công	Tên: Trần B, SĐT: 0909xxx, Phòng: Standard, Ngày: 20/07/2025, Số đêm: 3	Hiển thị “Thêm đơn đặt phòng thành công” và lưu đơn vào hệ thống	Đạt
TC02	Thiếu thông tin khách	Thiếu số điện thoại	Hiển thị cảnh báo “Vui lòng nhập số điện thoại khách hàng”	Đạt
TC03	Gửi yêu cầu thiếu token xác thực	Không có token	Phản hồi lỗi “Chưa đăng nhập”	Đạt

Bảng 4.21: Bảng kiểm thử chức năng thêm đơn đặt phòng

STT	Trường hợp kiểm thử	Đầu vào	Kết quả mong đợi	Đánh giá
TC01	Thêm loại phòng thành công	Tên: Deluxe; Diện tích: 35; Số khách: 2; Giá: 1.000.000; Mô tả: Phòng rộng; Ảnh: đã chọn	Thông báo "Thêm loại phòng thành công", form đóng, danh sách cập nhật	Đạt
TC02	Thiếu tên loại phòng	Tên: [rỗng]; các trường khác hợp lệ	Hiển thị cảnh báo “Tên loại phòng không được để trống”	Đạt
TC03	Nhập giá tiền không hợp lệ	Giá tiền: "một triệu"	Cảnh báo lỗi định dạng số	Đạt
TC04	Tên loại phòng trùng	Tên: Deluxe (đã có trong hệ thống)	Thông báo lỗi “Loại phòng này đã tồn tại”	Đạt

Bảng 4.22: Bảng kiểm thử chức năng thêm loại phòng

4.5 Triển khai

Ứng dụng được xây dựng và thử nghiệm trên môi trường cục bộ (*localhost*) theo mô hình **client-server**, với kiến trúc tách biệt giữa **frontend**, **backend** và **module học máy**. Việc triển khai này giúp tối ưu hóa chi phí và hỗ trợ sinh viên trong quá trình phát triển, kiểm thử tính năng trước khi triển khai thực tế.

4.5.1 Mô hình triển khai

- **Frontend:** Phát triển bằng ReactJS kết hợp với Vite để tăng tốc quá trình *build* và chạy ứng dụng.
- **Backend:** Sử dụng Node.js và Express để xây dựng các API xử lý nghiệp vụ

liên quan đến khách sạn.

- **Machine Learning Module:** Được tách thành một dịch vụ riêng biệt, sử dụng Flask (Python) để phục vụ mô hình dự đoán lượng khách hàng.
- **Database:** Hệ quản trị cơ sở dữ liệu MySQL, dùng để lưu trữ toàn bộ thông tin về người dùng, khách sạn, đặt phòng, v.v.

4.5.2 Thiết bị triển khai thử nghiệm

Ứng dụng được triển khai và kiểm thử trên máy tính cá nhân Acer Nitro 5 với cấu hình như sau:

Thành phần	Thông tin chi tiết
Thiết bị	Laptop Acer Nitro 5
Hệ điều hành	Windows 11 Home 64-bit
CPU	Intel Core i5-11400H
RAM	16GB
Ổ cứng	SSD 512GB
Công nghệ đồ họa	NVIDIA GeForce RTX 3050

Bảng 4.23: Cấu hình thiết bị triển khai thử nghiệm

4.5.3 Kết quả triển khai

Trong quá trình triển khai trên môi trường cục bộ, hệ thống đã được kiểm thử toàn diện về chức năng, hiệu suất và tính ổn định. Các chức năng chính như đặt phòng, quản lý khách sạn, xử lý thanh toán, và dự đoán lượng khách bằng học máy đều hoạt động đúng với kỳ vọng và cho kết quả khả quan.

Hệ thống được kiểm thử bởi chính nhóm sinh viên phát triển dưới nhiều vai trò khác nhau như người dùng, nhân viên và quản trị viên, nhằm kiểm tra toàn diện theo từng luồng nghiệp vụ.

Một số kết quả nổi bật

- Đáp ứng đầy đủ các yêu cầu chức năng đã đặt ra từ đầu đề án: từ đăng ký, đăng nhập, đặt phòng, thanh toán bằng VNPAY, sử dụng voucher, quản lý người dùng và phòng, đến tích hợp học máy để dự đoán lượng khách.
- Giao diện thân thiện, dễ sử dụng, phản hồi rõ ràng, hỗ trợ đầy đủ các thao tác CRUD.
- Mô hình học máy (dự đoán lượng khách) hoạt động ổn định, thời gian phản hồi trung bình nhỏ hơn 1 giây.
- Khả năng xử lý đồng thời nhiều yêu cầu trong quá trình thử nghiệm trên nhiều tab trình duyệt ổn định, không gặp lỗi bất ngờ.

- Tốc độ phản hồi API nhanh, trung bình từ 100–300ms đối với các truy vấn thường; các tác vụ xử lý ảnh (như hiển thị ảnh phòng) hoạt động mượt mà.
- Dữ liệu được lưu trữ và truy xuất ổn định trong MySQL, không có lỗi mất dữ liệu hay trùng lặp trong quá trình kiểm thử.

Đánh giá tổng thể: Kết quả thử nghiệm cho thấy ứng dụng đã giải quyết tốt các yêu cầu đề ra, cả về mặt **chức năng**, **hiệu năng** lẫn **trải nghiệm người dùng**. Việc tổ chức kiến trúc tách biệt giữa frontend, backend và mô-đun học máy giúp việc bảo trì và mở rộng hệ thống trong tương lai trở nên dễ dàng hơn.

4.6 Tổng kết

Chương 4 đã trình bày chi tiết quá trình thiết kế và triển khai hệ thống quản lý và đặt phòng khách sạn thông minh. Từ việc lựa chọn kiến trúc phần mềm theo mô hình MVC đến xây dựng các mô hình UML minh họa cấu trúc hệ thống, tất cả đều nhằm đảm bảo tính rõ ràng, khả năng mở rộng và dễ bảo trì trong suốt vòng đời ứng dụng.

Thông qua việc áp dụng các công nghệ hiện đại như ReactJS, Express, Sequelize ORM, MySQL, cùng với mô-đun học máy được phát triển bằng Flask, hệ thống không chỉ đáp ứng tốt các chức năng nghiệp vụ mà còn mang lại trải nghiệm người dùng mượt mà và hiệu quả. Phần kiểm thử và triển khai thực tế đã chứng minh rằng hệ thống hoạt động ổn định, đáng tin cậy và có khả năng thích ứng với nhu cầu mở rộng trong tương lai.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Sau quá trình nghiên cứu, phân tích yêu cầu và triển khai hệ thống quản lý – đặt phòng khách sạn, em nhận thấy rằng một hệ thống hoàn thiện không chỉ cần đáp ứng đầy đủ các chức năng nghiệp vụ, mà còn đòi hỏi những giải pháp kỹ thuật và thiết kế sáng tạo nhằm đảm bảo hiệu quả hoạt động, khả năng mở rộng và tính thực tiễn khi áp dụng vào thực tế.

Chương này trình bày các đóng góp nổi bật mà em đã thực hiện trong quá trình xây dựng đồ án, bao gồm việc giải quyết các bài toán nghiệp vụ đặc thù, đề xuất và ứng dụng các mô hình học máy phục vụ dự đoán lượng khách, thiết kế giao diện người dùng theo các nguyên tắc chuẩn. Mỗi nội dung được trình bày theo cấu trúc có vấn đề cụ thể, giải pháp được lựa chọn và kết quả đạt được.

5.1 Nghiệp vụ cho hệ thống booking và quản lý khách sạn

5.1.1 Giới thiệu bài toán

Trong lĩnh vực quản lý và vận hành khách sạn, hệ thống thông tin cần xử lý nhiều nghiệp vụ phức tạp, có tính liên kết chặt chẽ và yêu cầu độ chính xác cao. Các nghiệp vụ quan trọng bao gồm: quản lý thông tin phòng và loại phòng, tiếp nhận và xử lý đơn đặt phòng, tổ chức cung cấp các dịch vụ đi kèm, quản lý mã giảm giá (voucher), thống kê doanh thu, cũng như hỗ trợ quy trình khách hàng tự đặt phòng và đăng ký dịch vụ. Việc tích hợp nhiều chức năng này vào một hệ thống thống nhất nhằm mục tiêu nâng cao hiệu suất quản lý và giảm thiểu sai sót thủ công trong quy trình vận hành.

5.1.2 Giải pháp

Để hiện thực hóa các yêu cầu nghiệp vụ nêu trên, hệ thống được thiết kế và triển khai các chức năng sau:

- **Quản lý phòng và loại phòng:** Hệ thống cho phép quản trị viên thêm, sửa, xóa thông tin các loại phòng, gán tiện ích đi kèm, cập nhật trạng thái phòng. Điều này giúp đơn giản hóa công tác kiểm soát cơ sở vật chất.
- **Quản lý đơn đặt phòng:** Nhân viên hoặc khách hàng có thể tạo và xử lý các đơn đặt phòng. Đơn đặt có thể có trạng thái như *chờ xác nhận*, *đã xác nhận*, *đã thanh toán* hoặc *đã hủy*. Hệ thống cũng hỗ trợ tính năng tự động hủy đơn nếu khách không thanh toán đúng hạn.
- **Quản lý dịch vụ:** Lễ tân có thể xác nhận các đơn đặt dịch vụ. Hệ thống hỗ trợ kiểm tra lịch sử đơn và tình trạng xử lý để đảm bảo không bị bỏ sót.

- **Quản lý voucher:** Admin có thể tạo voucher giảm giá, giới hạn thời gian sử dụng. Hệ thống kiểm tra tính hợp lệ của voucher khi khách hàng áp dụng trong quá trình thanh toán.
- **Thông kê doanh thu:** Cung cấp các biểu đồ và bảng thống kê doanh thu theo thời gian (tuần, tháng, năm). Chức năng này hỗ trợ ban quản lý khách sạn đưa ra các quyết định chiến lược.
- **Đặt phòng và đặt dịch vụ:** Giao diện đặt phòng và dịch vụ trực tuyến cho phép khách hàng lựa chọn theo nhu cầu, xem chi tiết giá, diện tích, tiện ích đi kèm, và thực hiện thanh toán. Quá trình này diễn ra nhanh chóng và thân thiện với người dùng.

5.1.3 Kết quả đạt được

Qua quá trình thiết kế và triển khai các nghiệp vụ trên, hệ thống đã chứng minh khả năng đáp ứng tốt các yêu cầu thực tế trong quản lý khách sạn. Một số kết quả nổi bật bao gồm:

- Tăng tính hiệu quả trong quy trình đặt phòng và xử lý dịch vụ.
- Giảm thiểu sai sót thủ công nhờ hệ thống hóa quy trình nghiệp vụ.
- Hỗ trợ linh hoạt cho cả khách hàng và nhân viên trong các thao tác sử dụng.
- Cải thiện khả năng giám sát và đánh giá hiệu quả kinh doanh thông qua các công cụ thống kê trực quan.

5.2 Ứng dụng học máy trong dự đoán lượng khách

5.2.1 Giới thiệu bài toán

Trong lĩnh vực kinh doanh khách sạn, việc dự đoán trước số lượng khách lưu trú theo từng ngày, từng tuần hay từng giai đoạn trong năm là một nhu cầu thiết yếu để tối ưu hóa vận hành. Nếu không có khả năng dự báo chính xác, khách sạn có thể rơi vào tình trạng hoặc là quá tải (nếu nhu cầu tăng đột biến mà không chuẩn bị kịp), hoặc là hoạt động kém hiệu quả do phòng trống nhiều, nhân sự dư thừa, và dịch vụ không được sử dụng hết. Cả hai tình huống này đều ảnh hưởng trực tiếp đến doanh thu và trải nghiệm khách hàng.

Ngoài ra, việc dự đoán lượng khách cũng có ý nghĩa quan trọng trong việc lập kế hoạch marketing, tổ chức các chương trình ưu đãi theo mùa, và phân bổ nhân sự phù hợp vào các khung thời gian cao điểm. Vì lý do đó, em quyết định nghiên cứu và tích hợp một mô hình học máy (machine learning) vào hệ thống nhằm dự đoán lượng khách theo ngày dựa trên dữ liệu đặt phòng lịch sử và các yếu tố liên quan.

5.2.2 Giải pháp

Để giải quyết bài toán này, hệ thống đã xây dựng một **module học máy độc lập** sử dụng ngôn ngữ lập trình Python và thư viện `scikit-learn`. Dữ liệu đầu vào được thu thập từ hệ thống đặt phòng, bao gồm các trường thông tin sau:

- Ngày nhận phòng
- Ngày trong tuần
- Tháng
- Có phải ngày lễ không
- Thời tiết
- Nhiệt độ
- Có mã giảm giá không
- Lượng khách đặt trong ngày

Tiền xử lý dữ liệu:

- Áp dụng kỹ thuật *one-hot encoding* cho các biến phân loại như ngày trong tuần, thời tiết.

Mô hình sử dụng:

- Mô hình Random Forest: Mô hình học máy phi tuyến tính này giúp xử lý tốt dữ liệu có nhiều biến đầu vào và mối quan hệ phức tạp. Ưu điểm của Random Forest là ít bị overfitting và dễ huấn luyện. Mô hình này cho kết quả tương đối ổn định.
- Mô hình XGBoost: Với khả năng tối ưu hóa và học sâu trên các mẫu khó, XGBoost được sử dụng để cải thiện độ chính xác. Em đã tinh chỉnh các tham số như số cây, learning rate, và max_depth để đạt được kết quả tốt hơn.
- Mô hình SARIMA: Đây là mô hình dự báo chuỗi thời gian có tính đến yếu tố mùa vụ. SARIMA hoạt động tốt trên dữ liệu có chu kỳ rõ ràng như lượng khách theo tuần hoặc tháng. Dù không xử lý được nhiều biến đầu vào như XGBoost nhưng lại phù hợp để mô phỏng xu hướng thời gian dài hạn.

Tích hợp vào hệ thống:

- Module học máy được triển khai dưới dạng một RESTful API sử dụng framework `Flask`, có thể nhận yêu cầu từ backend và trả về số lượng khách dự đoán theo từng ngày.

5.2.3 Kết quả đạt được

Việc tích hợp module học máy vào hệ thống đã giúp người quản lý khách sạn:

- Chủ động điều phối phòng và dịch vụ vào các ngày cao điểm.
- Tối ưu hóa lịch trực của nhân viên.
- Tăng hiệu quả trong các chiến dịch ưu đãi, marketing theo mùa.

5.3 Áp dụng các nguyên tắc thiết kế vào giao diện người dùng

5.3.1 Giới thiệu vấn đề

Một hệ thống quản lý và đặt phòng khách sạn không chỉ cần chính xác về mặt chức năng mà còn phải đảm bảo trải nghiệm người dùng (UX) mượt mà và giao diện người dùng (UI) trực quan, dễ sử dụng. Vấn đề đặt ra là làm sao để người dùng (bao gồm cả khách hàng và nhân viên) có thể nhanh chóng thao tác, tìm kiếm thông tin và thực hiện hành động một cách hiệu quả, đặc biệt trong bối cảnh hệ thống có nhiều phân hệ như đặt phòng, quản lý dịch vụ, hóa đơn, voucher, v.v.

5.3.2 Giải pháp thiết kế giao diện

Để giải quyết bài toán trên, hệ thống đã áp dụng một số nguyên tắc thiết kế UI/UX quan trọng trong quá trình xây dựng giao diện bằng ReactJS như sau:

1. Nguyên tắc phân cấp thị giác (Visual Hierarchy)

Sử dụng kích thước font chữ, màu sắc nổi bật, và khoảng cách rõ ràng để làm nổi bật các thành phần quan trọng như nút “Đặt phòng”, “Thêm dịch vụ”, “Xác nhận”.

Ví dụ: Trong trang đặt phòng, thông tin giá và loại phòng được hiển thị lớn và đậm hơn so với các thông tin phụ như diện tích hay mô tả.

2. Tính nhất quán (Consistency)

Giao diện sử dụng thống nhất một bộ màu sắc và thành phần giao diện (components) xuyên suốt toàn bộ hệ thống.

Màu xanh biểu thị thao tác chính, đỏ cho hành động xóa, hiệu ứng mờ cho trạng thái bị vô hiệu hóa.

3. Phản hồi người dùng (Feedback)

Cung cấp phản hồi trực quan cho người dùng thông qua các biểu tượng loading, toast thông báo.

Ví dụ: Sau khi đặt phòng thành công, hệ thống hiển thị thông báo “Đặt phòng thành công” kèm biểu tượng xác nhận.

4. Thiết kế hướng hành động (Clear Call-to-Action)

Các nút hành động được thiết kế nổi bật và đặt ở vị trí người dùng dễ nhận

biết nhất.

Ví dụ: Nút “Xác nhận thanh toán” luôn được đặt ở cuối biểu mẫu, có kích thước lớn, dễ thao tác.

5.3.3 Kết quả đạt được

Việc áp dụng các nguyên tắc thiết kế UI/UX đã mang lại những cải thiện đáng kể trong quá trình sử dụng hệ thống:

- Tăng tốc độ thao tác nhờ bố cục hợp lý, rõ ràng.
- Giảm lỗi nhập liệu và thao tác nhờ phản hồi nhanh và hướng dẫn trực quan.
- Nâng cao mức độ hài lòng của cả khách hàng lẫn nhân viên trong các thử nghiệm nội bộ.
- Dễ dàng mở rộng hoặc tùy chỉnh giao diện trong tương lai nhờ kiến trúc component-based của React.

5.4 Tổng kết

Những nội dung trình bày trong chương này là phần kết tinh những nỗ lực và suy nghĩ chủ động của em trong suốt quá trình thực hiện đồ án. Từ việc xử lý các nghiệp vụ phức tạp trong quản lý khách sạn, xây dựng mô hình học máy để tăng tính thông minh cho hệ thống, cho đến việc áp dụng các nguyên tắc thiết kế giao diện hiện đại, tất cả đều hướng đến mục tiêu tạo ra một hệ thống vừa mạnh mẽ về mặt kỹ thuật, vừa thân thiện với người dùng.

Thông qua những giải pháp được nêu trong chương này, em mong muốn góp phần khẳng định tính khả thi và giá trị thực tiễn của hệ thống trong môi trường vận hành thực tế. Đây không chỉ là những bước tiến kỹ thuật mà còn là kết quả của quá trình học hỏi, thích nghi và phát triển tư duy lập trình – thiết kế một cách có hệ thống và hiệu quả.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong quá trình nghiên cứu và phát triển hệ thống đặt phòng và quản lý khách sạn, em có tham khảo một số nền tảng phổ biến như *Booking.com*, *Agoda*, cũng như một số hệ thống quản lý khách sạn nội bộ. Các hệ thống này thường có quy mô lớn, tích hợp đa dạng chức năng từ đặt phòng, thanh toán, đến quản lý chuỗi khách sạn và hỗ trợ nhiều loại người dùng.

So với các hệ thống thương mại, sản phẩm của em tuy còn giới hạn về quy mô và chưa có các tính năng nâng cao như tích hợp thanh toán quốc tế, hệ thống phân phối toàn cầu (GDS), hay hệ sinh thái di động, nhưng lại có một số ưu điểm nổi bật trong phạm vi sinh viên thực hiện được:

- Hệ thống có khả năng quản lý toàn bộ quy trình: từ đặt phòng, đặt dịch vụ, đến quản lý phòng – dịch vụ – voucher – báo cáo doanh thu.
- Tích hợp mô hình học máy để dự đoán lượng khách theo ngày, hỗ trợ ra quyết định và điều phối nguồn lực hiệu quả. Đây là điểm mới, ít xuất hiện trước đây.
- Giao diện người dùng trực quan, áp dụng các nguyên tắc thiết kế như tính nhất quán (consistency), phản hồi (feedback), ..., góp phần nâng cao trải nghiệm sử dụng.

6.1.1 Tự đánh giá kết quả thực hiện đồ án

a, Những gì đã làm được

- Phân tích đầy đủ yêu cầu người dùng, thiết kế hệ thống theo kiến trúc phân tầng rõ ràng (MVC kết hợp RESTful API).
- Xây dựng hệ thống hoàn chỉnh và triển khai cục bộ, bao gồm các module chức năng chính: đặt phòng, quản lý phòng – dịch vụ – hóa đơn – voucher – thống kê doanh thu.
- Tích hợp thành công các mô hình học máy (Random Forest, XGBoost, SARIMA) cho bài toán dự đoán lượng khách.
- Thực hiện kiểm thử logic và chức năng đối với các thao tác quan trọng: đặt phòng, thêm dịch vụ, tạo voucher.
- Áp dụng nguyên tắc thiết kế giao diện hiện đại, mang lại trải nghiệm người dùng thân thiện.

b, Những gì chưa làm được

- Hệ thống chưa được triển khai trên môi trường thực tế (cloud, production), chưa kiểm chứng được hiệu năng và khả năng mở rộng thực tế.
- Việc đánh giá hiệu quả mô hình học máy còn giới hạn do thiếu dữ liệu lớn và phong phú.

6.1.2 Đóng góp nổi bật

- Kết hợp giữa hệ thống quản lý nghiệp vụ và học máy, giúp hệ thống không chỉ dừng lại ở CRUD đơn thuần mà còn có khả năng học từ dữ liệu và đưa ra dự đoán.
- Thiết kế linh hoạt, dễ mở rộng.
- Kiến trúc phần mềm rõ ràng, dễ bảo trì, dễ triển khai lại trên môi trường khác.
- Tự xây dựng toàn bộ hệ thống từ frontend, backend đến API, cơ sở dữ liệu và module học máy, thể hiện năng lực phát triển toàn diện.

6.1.3 Bài học kinh nghiệm rút ra

- Hiểu rõ bài toán nghiệp vụ là điều kiện tiên quyết để thiết kế đúng kiến trúc và mô hình dữ liệu.
- Quản lý phiên bản mã nguồn và kiểm thử liên tục là yếu tố quan trọng để tránh lỗi và đảm bảo tiến độ dự án.
- Việc tích hợp học máy vào hệ thống thực tế yêu cầu kỹ năng kết nối đa công nghệ: từ xử lý dữ liệu, huấn luyện mô hình, đến triển khai API.
- Làm việc theo hướng “đóng gói từng phần – kiểm thử từng bước” giúp giảm rủi ro và linh hoạt trong việc sửa lỗi logic.
- Đặt mình vào vị trí của người dùng (khách hàng, lễ tân, quản lý) giúp có cái nhìn thực tế hơn về UX/UI và cải tiến hệ thống sát với nhu cầu thực tế.

6.2 Hướng phát triển

6.2.1 Hoàn thiện các chức năng hiện tại

Mặc dù hệ thống đã đáp ứng phần lớn các yêu cầu nghiệp vụ cơ bản trong quản lý và đặt phòng khách sạn, vẫn còn một số điểm cần hoàn thiện để tăng tính thực tiễn và độ tin cậy:

- **Triển khai hệ thống trên môi trường cloud** như AWS, Heroku hoặc Vercel để kiểm tra khả năng hoạt động thực tế, phục vụ truy cập đa người dùng.
- **Hoàn thiện kiểm thử tự động** cho toàn bộ API backend (bằng Jest/Supertest hoặc Postman Collection), nhằm đảm bảo độ ổn định khi nâng cấp hoặc sửa

lỗi hệ thống.

- **Cải tiến trải nghiệm người dùng (UI/UX)** bằng cách bổ sung hiệu ứng chuyển trạng thái mượt mà, biểu tượng rõ ràng hơn, và hệ thống thông báo minh bạch cho các thao tác.

6.2.2 Nâng cấp và mở rộng chức năng

Sau khi hoàn thiện các chức năng cơ bản, hệ thống có thể được mở rộng theo các hướng sau để phục vụ tốt hơn nhu cầu thực tế của các khách sạn:

- **Tích hợp bản đồ (Map API):** Cho phép người dùng xem vị trí khách sạn và các dịch vụ xung quanh bằng Google Maps hoặc Leaflet.
- **Ứng dụng di động:** Phát triển phiên bản mobile sử dụng React Native hoặc Flutter để người dùng có thể thao tác đặt phòng và quản lý tiện lợi trên thiết bị di động.
- **Mở rộng học máy:**
 - Nâng cao chất lượng dự báo bằng cách huấn luyện mô hình trên tập dữ liệu lớn hơn và đa chiều hơn (theo mùa, sự kiện địa phương).
 - Áp dụng thêm kỹ thuật học sâu (Deep Learning), đặc biệt là mạng LSTM cho bài toán chuỗi thời gian nếu có đủ dữ liệu.
 - Tự động hóa các tác vụ quản lý như đề xuất giá phòng theo nhu cầu thị trường, nhắc tin nhắc khách nhận phòng hoặc thanh toán.
- **Hỗ trợ đa ngôn ngữ:** Bổ sung giao diện tiếng Anh hoặc các ngôn ngữ phổ biến khác nhằm phục vụ khách quốc tế.

6.2.3 Tổng kết

Với các định hướng như trên, hệ thống có tiềm năng phát triển thành một giải pháp phần mềm quản lý khách sạn toàn diện và thông minh, không chỉ phục vụ nội bộ mà còn hướng đến khách hàng bên ngoài. Sự kết hợp giữa kỹ thuật lập trình phần mềm và học máy chính là điểm nhấn tạo nên giá trị ứng dụng thực tiễn cao trong bối cảnh chuyển đổi số mạnh mẽ hiện nay.

TÀI LIỆU THAM KHẢO

- [1] VNPay, *VNPay Sandbox API Documentation*. [Online]. Available: <https://sandbox.vnpayment.vn/apis/vnpay-demo/> (visited on 05/30/2025).
- [2] Võ Hồng Hải, *Học máy cơ bản và ứng dụng*. NXB Đại học Quốc gia TP.HCM, 2020.
- [3] Martin Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [4] Tianqi Chen and Carlos Guestrin, “XGBoost: A Scalable Tree Boosting System,” in **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**, 2016, pp. 785–794.